UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA: Informatica Romana

LUCRARE DE LICENȚĂ

Sistem de detecție a anomaliilor în traficul de rețea bazat pe un algoritm de Machine Learning
Network traffic intrusion detection system

Conducător științific:
Conf. Univ. Dr. Bufnea Darius

Absolvent:
Banea Dragos Alexandru

2026

Abstract

This paper presents the design and implementation of a network intrusion detection system based on Machine Learning techniques. Motivated by the limitations of classical signature-based systems, the proposed solution uses anomaly detection algorithms - Random Forest, Isolation Forest, and One-Class SVM - capable of identifying unknown attacks without requiring manual rule updates.

The system implements a complete pipeline: traffic capture, extraction of statistical and temporal features from network flows, model training on the CICIDS2017 dataset, and real-time classification of new flows. Experimental results demonstrate high accuracy in detecting DoS, Brute Force, and Port Scanning attacks, while also highlighting the challenges related to class imbalance and concept drift.

Cuprins

Abstract **1**

1 Introducere **6**

2 Fundamente de rețelistică **8**

2.1 Modele de rețea 9

2.1.1 Modelul OSI 9

2.1.2 Modelul TCP/IP 11

2.2 Protocoale de comunicație 12

2.2.1 IP, TCP, UDP 12

2.2.2 Porturi și sesiuni 13

2.3 Tipuri de trafic în rețea 14

2.3.1 Trafic normal 14

2.3.2 Trafic anormal 15

2.4 Fluxuri de rețea (network flows) 15

2.4.1 Definiție 15

2.4.2 Caracteristici (features) 16

3 Securitate rețea și sisteme IDS **18**

3.1 Tipuri de atacuri informatice 18

3.1.1 DoS / DDoS 18

3.1.2 Brute force 19

3.1.3 Port scanning 19

3.1.4 Malware 20

3.2 Sisteme de detecție a intruziunilor (IDS) 20

3.2.1 Definiție 20

3.2.2 Rol 21

3.3 Clasificarea IDS 21

3.3.1 Signature-based 21

3.3.2 Anomaly-based 22

3.3.3 Rule-based 22

3.4 Tipuri de IDS . . . 23
3.4.1 NIDS (Network-based) . . . 23
3.4.2 HIDS (Host-based) . . . 23
3.5 Limitările sistemelor clasice . . . 24
3.5.1 Imposibilitatea detectării atacurilor noi . . . 24
3.5.2 Dependența de semnături . . . 24
3.5.3 Problema fals-pozitivelor . . . 25

4 Machine Learning pentru detecția anomaliilor 26
4.1 Introducere în Machine Learning . . . 26
4.1.1 Supervised learning . . . 26
4.1.2 Unsupervised learning . . . 27
4.1.3 Semi-supervised learning . . . 27
4.2 Detecția anomaliilor . . . 28
4.2.1 Definiție . . . 28
4.2.2 Tipuri de anomalii . . . 29
4.2.3 Aplicații în securitate . . . 29
4.3 Algoritmi utilizați . . . 30
4.3.1 Random Forest . . . 30
4.3.2 One-Class SVM . . . 31
4.3.3 Isolation Forest . . . 31
4.4 Feature engineering în trafic de rețea . . . 32
4.4.1 Caracteristici statistice . . . 32
4.4.2 Caracteristici temporale . . . 32
4.4.3 Caracteristici bazate pe flux . . . 33
4.5 Probleme specifice ML în securitate . . . 33
4.5.1 Dezechilibru de clase . . . 33
4.5.2 Concept drift . . . 34
4.5.3 False positives în context ML . . . 34
4.6 Dataset-uri pentru IDS . . . 35
4.6.1 NSL-KDD . . . 35
4.6.2 CICIDS2017 . . . 35
4.6.3 Alte dataset-uri . . . 36
4.7 Preprocesarea datelor . . . 36
4.7.1 Curățare date . . . 36
4.7.2 Normalizare . . . 37
4.7.3 Selecția caracteristicilor . . . 37
4.8 Metrici de evaluare . . . 37
4.8.1 Accuracy . . . 38

4.8.2 Precision . . . 38
4.8.3 Recall . . . 38
4.8.4 F1-score . . . 38
4.8.5 ROC-AUC . . . 39
4.9 Validarea modelelor . . . 39
4.9.1 Train/Test split . . . 39
4.9.2 Cross-validation . . . 39

5 Prezentarea aplicației propuse 41
5.1 Analiza cerințelor . . . 41
5.1.1 Cerințe funcționale . . . 41
5.1.2 Cerințe nefuncționale . . . 42
5.2 Arhitectura sistemului . . . 44
5.2.1 Componente principale . . . 44
5.2.2 Fluxul datelor . . . 45
5.3 Tehnologii utilizate . . . 45
5.4 Senzorul de rețea . . . 45
5.4.1 Capturarea pachetelor . . . 45
5.4.2 Agregarea fluxurilor și extragerea caracteristicilor . . . 46
5.5 Serviciul de Machine Learning . . . 46
5.5.1 Preprocesarea și antrenarea modelelor . . . 46
5.5.2 Endpoint-ul de predicție . . . 47
5.6 Backend-ul aplicației . . . 47
5.6.1 Pipeline-ul de analiză . . . 47
5.6.2 Generarea și clasificarea alertelor . . . 48
5.7 Interfața utilizator . . . 49
5.7.1 Dashboard . . . 49
5.7.2 Live Monitor . . . 49
5.7.3 Gestionarea alertelor . . . 50
5.7.4 Metricile modelului . . . 51
5.8 Evaluarea sistemului . . . 51
5.8.1 Rezultate experimentale . . . 51
5.8.2 Analiza performanței . . . 52

6 Concluzii teoretice și direcții 54
6.1 Sinteza metodelor prezentate . . . 54
6.2 Limitări . . . 54
6.3 Testarea sistemului . . . 55
6.3.1 Teste unitare - backend (JUnit 5 + Mockito) . . . 55
6.3.2 Teste unitare - senzorul de rețea (pytest) . . . 56

6.3.3 Teste de integrare - serviciul AI (pytest + FastAPI TestClient) 56
6.4 Direcții viitoare 57
6.4.1 Deep Learning 57
6.4.2 Sisteme în timp real la scară 58

Capitolul 1

Introducere

În contextul dezvoltării accelerate a tehnologiilor informaționale și al extinderii continue a rețelelor de calculatoare, securitatea cibernetică a devenit o preocupare majoră atât pentru organizații, cât și pentru utilizatorii individuali. Creșterea volumului de date transferate și diversificarea serviciilor oferite prin intermediul rețelelor au condus la apariția unor amenințări din ce în ce mai sofisticate, care pot compromite confidențialitatea, integritatea și disponibilitatea informațiilor (atacuri de tip DoS, tentative de acces neautorizat sau răspândirea de malware) [1].

Pentru a face față acestor amenințări, au fost dezvoltate diverse metode și tehnologii de securitate, printre care se numără și sistemele de detecție a intruziunilor (Intrusion Detection Systems - IDS) [1, 2]. Aceste sisteme au rolul de a monitoriza traficul de rețea și de a identifica activități suspecte sau anormale care ar putea indica prezența unui atac cibernetic. Sistemele tradiționale de tip IDS, bazate pe semnături sau reguli predefinite, prezintă însă limitări semnificative, în special în detectarea atacurilor necunoscute sau a celor care au o evoluție rapidă. Acest lucru se datorează faptului că aceste sisteme se bazează pe modele statice și nu se pot adapta rapid la schimbările din rețea sau la noi tipuri de atacuri.

În acest context, utilizarea tehnicilor de Machine Learning pentru detecția anomaliilor în traficul de rețea reprezintă o direcție promițătoare pentru îmbunătățirea capacităților de detecție a sistemelor IDS. Prin analizarea automată a datelor de trafic și identificarea tiparelor anormale, modelele de învățare automată pot detecta atacuri noi sau necunoscute, oferind astfel o creștere semnificativă a nivelului de securitate al rețelelor de calculatoare.

Cercetările recente în domeniu au demonstrat potențialul metodelor de învățare automată în detecția intruziunilor [2]. Abordări bazate pe algoritmi precum Random Forest, Support Vector Machines sau rețele neuronale au obținut rezultate promițătoare pe dataset-uri de referință precum KDD Cup 99, NSL-KDD și CICIDS2017. Cu toate acestea, majoritatea soluțiilor existente sunt evaluate în condiții de laborator și nu adresează în mod direct detectarea anomaliilor noi, necunoscute la momentul antrenării. Lucrarea de față își propune să trateze tocmai această limitare, combinând tehnici de detecție bazată pe anomalii cu un flux de procesare adaptat traficului real de rețea.

În această lucrare, propunem dezvoltarea unui sistem de detectare a intruziunilor bazat pe

tehnici de Machine Learning, care să poată analiza traficul de rețea în timp real și să identifice anomalii care ar putea indica prezența unui atac cibernetic. Sistemul este conceput pentru a colecta date din rețea, a extrage caracteristici relevante și a utiliza un model de învățare automată pentru a clasifica și detecta anomaliile în traficul de rețea. În cazul identificării unei potențiale anomalii, sistemul va genera o alertă pentru a informa administratorii de rețea și pentru a permite luarea unor măsuri rapide de remediere.

Scopul principal al acestei lucrări este de a analiza metodele existente de detecție a intruziunilor și de a propune o soluție mai eficientă și adaptabilă, bazată pe tehnici de Machine Learning. De asemenea, lucrarea își propune să evidențieze avantajele și limitările acestor metode, precum și provocările asociate implementării unui astfel de sistem IDS. Obiectivele specifice ale lucrării sunt:

- Analiza comparativă a algoritmilor de Machine Learning (Random Forest, Isolation Forest, One-Class SVM) aplicați pe date de trafic de rețea;
- Evaluarea performanței sistemului pe dataset-ul public CICIDS2017, urmărind metrici precum Precizia, Recall-ul și F1-score-ul;
- Implementarea unui pipeline complet de procesare: captură de trafic, extragere de caracteristici, antrenare model și generare de alerte;
- Identificarea limitărilor abordării propuse și a direcțiilor de îmbunătățire ulterioară.

Lucrarea este structurată în cinci capitole. Capitolul 2 prezintă fundamentele rețelisticii necesare înțelegerii traficului analizat: modele de rețea, protocoale și tipuri de fluxuri. Capitolul 3 descrie amenințările informatice relevante și clasificarea sistemelor IDS existente, evidențiind limitările abordărilor clasice. Capitolul 4 introduce tehnicile de Machine Learning utilizate, cu accent pe detecția anomaliilor, algoritmii selectați și procesul de preprocesare a datelor. Capitolul 5 prezintă în detaliu arhitectura și implementarea sistemului propus, împreună cu rezultatele experimentale obținute. Capitolul 6 sintetizează contribuțiile lucrării și propune direcții de cercetare viitoare.

În timpul elaborării acestei lucrări, autorul a utilizat modele lingvistice generative, precum ChatGPT și Claude Code, pentru asistarea completării unor fragmente de cod, generarea unor versiuni inițiale ale codului și, ocazional, reformularea unor enunțuri pentru sporirea clarității. După utilizarea acestor instrumente, autorul a revizuit și editat conținutul generat și își asumă întreaga responsabilitate pentru forma finală și pentru conținutul tezei.

Capitolul 2

Fundamente de rețelistică

Termenul reţea de calculatoare reprezintă un sistem care interconectează mai multe dispozitive (calculatoare, imprimante, routere, etc.), permiţându-le să comunice între ele şi să partajeze resurse [3, 5].

Metodele de interconectare a reţelelor de calculatoare sunt într-o continuă dezvoltare, variind de cabluri fizice (Ethernet, fibră optică) la conexiuni wireless (Wi-Fi, WiMAX, Bluetooth). Aceste metode permit transferul de date între dispozitivele din reţea, facilitând comunicarea şi partajarea resurselor.

Împărţirea şi organizarea reţelelor de calculatoare se realizează după 2 criterii: tehnologia de interconectare şi aria geografică acoperită [3, 5]. În funcţie de tehnologia de transmisie, reţelele pot fi:

- Legături cu difuzare (broadcast) - unde datele sunt transmise către toate dispozitivele din reţea, iar fiecare dispozitiv decide dacă să preia sau să ignore datele.
- Legături punct-la-punct (point-to-point) - unde datele sunt transmise direct între două dispozitive, fără a fi difuzate către alte dispozitive din reţea. Este posibilă nevoia de trasee multiple pentru a ajunge la destinaţie, iar dispozitivele intermediare (routere) pot fi necesare pentru a direcţiona traficul.

În funcţie de aria geografică acoperită, reţelele pot fi:

- Reţele locale (LAN) - acoperă o zonă geografică restrânsă, cum ar fi o casă, un birou sau un campus. Aceste reţele sunt de obicei rapide şi au costuri reduse.
- Reţele metropolitane (MAN) - acoperă o zonă geografică mai extinsă, cum ar fi un oraș sau un district. Aceste reţele pot fi construite folosind tehnologii precum Ethernet-ul sau Fibră Optică.
- Reţele de arie largă (WAN) - acoperă o zonă geografică foarte extinsă, cum ar fi o ţară, un continent sau întreaga lume. Aceste reţele sunt de obicei construite folosind tehnologii precum satelitele sau linii de comunicație subacvatice.

2.1 Modele de rețea

Modelele de reţea reprezintă o componentă esenţială în domeniul comunicaţiilor între dispozitive, oferind un mod structurat de a descrie modul în care datele sunt transmise între sisteme. Acestea definesc un set de reguli şi mecanisme prin care dispozitivele interconectate pot comunica eficient, indiferent de diferenţele hardware sau software dintre ele.

Pentru a reduce complexitatea proiectării şi implementării reţelelor de calculatoare, este folosită o organizare sub forma unei serii de straturi sau niveluri, fiecare având responsabilităţi bine definte. Această organiyare permite separarea funcţionalităţilor şi facilitează dezvoltarea, implementarea şi întreţinerea reţelelor de calculatoare. Fiecare strat sau nivel are un rol specific în procesul de comunicare, iar interacţiunea dintre aceste straturi asigură funcţionarea corectă a reţelei.

În prezent, cele mai importante modele utilizate pentru descrierea comunicaţiei în reţele sunt modelul OSI (Open Systems Interconnection) şi modelul TCP/IP (Transmission Control Protocol/Internet Protocol) [5]. Aceste modele oferă o bază teoretică pentru înţelegerea modului în care datele sunt transmise între sisteme, precum şi pentru dezvoltarea de protocoale şi tehnologii de reţea.

2.1.1 Modelul OSI

Modelul se mai numeşte şi modelul ISO (International Standards Organization) şi a fost dezvoltat de către Organizaţia Internaţională pentru Standardizare (ISO) în anii 1980 [5]. Acesta este un model de referinţă folosit pentru a descrie modul în care are loc comunicarea între sisteme. Acest model este compus din 7 straturi(figura 2.1), fiecare având un rol specific în procesul de comunicare între sisteme. Straturile sunt:

- Stratul fizic (Physical Layer) - cel mai de jos nivel al modelului, responsabil pentru transmiterea efectivă a datelor prin mediul fizic, cum ar fi cablurile sau undele radio. Se ocupă de modul în care bitul 0 şi bitul 1 sunt reprezentați fizic, precum şi de sincronizarea semnalelor.

- Stratul de legătură de date (Data Link Layer) - asigură o conexiune fiabilă între două dispozitive adiacente, gestionând erorile de transmisie şi controlul fluxului.

- Stratul de reţea (Network Layer) - se ocupă de rutarea datelor între diferite reţele, asigurând că pachetele ajung la destinaţia corectă.

- Stratul de transport (Transport Layer) - oferă servicii de transport fiabile între dispozitive, gestionând segmentarea datelor şi controlul fluxului.

- Stratul de sesiune (Session Layer) - gestionează sesiunile de comunicare între aplicaţii, asigurând sincronizarea şi controlul dialogului.

- Stratul de prezentare (Presentation Layer) - se ocupă de formatarea datelor pentru a fi înţelese de aplicaţii, gestionând conversia datelor şi criptarea.
- Stratul de aplicaţie (Application Layer) - oferă servicii pentru aplicaţii, cum ar fi e-mail, transfer de fişiere sau navigare web.

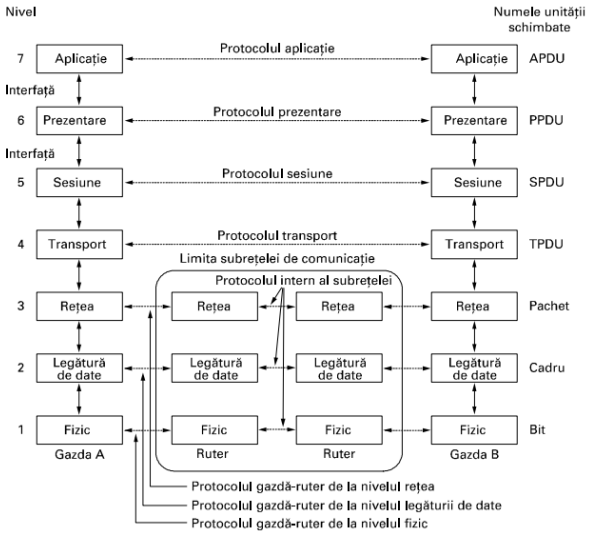


Figura 2.1: Modelul OSI [5]

Pentru a se ajunge la aceste 7 niveluri au fost aplicate următoarele principii [5]:

1. Un nivel trebuie creat atunci când este nevoie de un nivel de abstractizare diferit. - De exemplu, nivelul de transport a fost creat pentru a oferi servicii de transport fiabile, în timp ce nivelul de reţea a fost creat pentru a gestiona rutarea datelor între diferite reţele.

2. Fiecare nivel trebuie să îndeplinească un rol bine definit. - De exemplu, nivelul de legătură de date este responsabil pentru asigurarea unei conexiuni fiabile între două dispozitive adiacente, în timp ce nivelul de prezentare este responsabil pentru formatarea datelor pentru a fi înţelese de aplicaţii.

3. Funcţia fiecărui nivel trebuie aleasă acordându-se atenţie definirii de protocoale standardizate pe plan internaţional. - De exemplu, nivelul de transport utilizează protocoale standardizate precum TCP (Transmission Control Protocol) şi UDP (User Datagram Protocol) pentru a oferi servicii de transport fiabile între dispozitive.

4. Delimitarea nivelurilor trebuie făcută astfel încât să se minimizeze fluxul de informaţii prin interfeţe. - De exemplu, nivelul de reţea trebuie să se ocupe doar de rutarea datelor între diferite reţele, fără a se implica în detaliile legate de transportul datelor sau formatarea acestora.

5. Numărul de niveluri trebuie să fie suficient pentru a acoperi toate funcţiile necesare, dar nu atât de mare încât să devină dificil de înţeles sau implementat. - De exemplu, modelul

OSI a fost proiectat cu 7 niveluri pentru a acoperi toate funcţiile necesare în procesul de comunicare între sisteme, fără a deveni prea complex sau dificil de implementat.

2.1.2 Modelul TCP/IP

Modelul TCP/IP (Transmission Control Protocol/Internet Protocol) reprezintă un set de protocoale de comunicaţie utilizate pentru interconectarea sistemelor de calculatoare pe internet [5, 10]. Acesta defineste un set de reguli şi standarde care guvernează transmiterea datelor între dispozitive eterogene, asigurând interoperabilitatea între diferite sisteme şi reţele.

Spre deosebire de modelul OSI, care are un caracter teoretic şi conceptual, modelul TCP/IP a fost dezvoltat în mod practic pentru a răspunde nevoilor reale ale comunicaţiilor pe internet. Modelul TCP/IP este organizat în patru straturi, fiecare având un rol specific în procesul de transmitere a datelor între dispozitive. Straturile modelului TCP/IP sunt:

- Stratul de acces la reţea (Network Access Layer) - responsabil pentru gestiunea interacțiunii directe cu mediul fizic, incluzând aspecte legate de transmiterea datelor prin mediul fizic, gestionând aspecte precum adresarea fizică şi controlul accesului la mediu. Acest strat integrează funcționalități corespunzătoare straturilor fizic și legătură de date din modelul OSI și utilizează tehnologii precum Ethernet, Wi-Fi sau Fibră Optică pentru a asigura conectivitatea fizică între dispozitive. De asemenea, la acest nivel sunt utilizate adresele MAC (Media Access Control) pentru identificarea unică a dispozitivelor în cadrul unei rețele locale.

- Stratul de internet (Internet Layer) - se ocupă de adresarea logică și rutarea pachetelor de date între diferite rețele interconectate, asigurând un traseu optim pentru transmiterea datelor. Acest strat utilizează protocolul IP (Internet Protocol) pentru a gestiona adresele IP și pentru a asigura livrarea pachetelor către destinația corectă, indiferent de rețelele intermediare prin care trec.

- Stratul de transport (Transport Layer) - oferă servicii de transport fiabile între dispozitive, gestionând segmentarea datelor și controlul fluxului. Acest strat utilizează protocoale precum TCP (Transmission Control Protocol) pentru a asigura livrarea fiabilă a datelor și UDP (User Datagram Protocol) pentru a oferi un serviciu de transport mai rapid, dar fără garanții de livrare.

- Stratul aplicație (Application Layer) - reprezintă nivelul superior al modelului TCP/IP și oferă interfața directă între utilizator și rețea. Acesta include protocoalele necesare pentru furnizarea serviciilor de rețea utilizatorilor finali. Protocoalele din acest strat includ HTTP (Hypertext Transfer Protocol) pentru navigarea web, FTP (File Transfer Protocol) pentru transferul de fișiere, SMTP (Simple Mail Transfer Protocol) pentru trimiterea de e-mailuri și DNS (Domain Name System) pentru rezolvarea numelor de domenii în adrese IP.

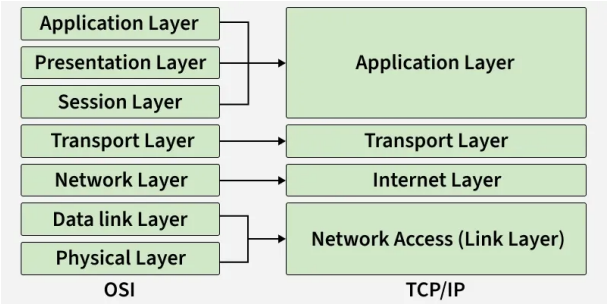


Figura 2.2: Modelul TCP/IP in relație cu modelul OSI [10]

Deși ambele modele descriu același proces de comunicare în rețea, există diferențe esențiale între ele. Modelul OSI, cu cele 7 straturi ale sale, are un caracter predominant teoretic și educațional, oferind o descriere detaliată și riguroasă a fiecărei funcții din comunicarea în rețea. Modelul TCP/IP, în schimb, a fost conceput cu un scop practic, condensând cele 7 straturi OSI în doar 4, prin unificarea unor funcții înrudite: stratul de aplicație al TCP/IP înglobează straturile de sesiune, prezentare și aplicație din OSI, iar stratul de acces la rețea corespunde straturilor fizic și legătură de date.

O altă diferență importantă constă în ordinea apariției: modelul OSI a fost definit înaintea protocoalelor pe care le descrie, urmărind să ofere un cadru universal independent de implementare. Modelul TCP/IP, dimpotrivă, a fost derivat din protocoalele deja existente ale internetului, ceea ce explică adoptarea sa largă în practică. În concluzie, OSI este modelul de referință folosit pentru înțelegerea și documentarea rețelelor, în timp ce TCP/IP reprezintă standardul efectiv utilizat în comunicațiile moderne.

2.2 Protocoale de comunicație

Protocoalele de comunicație reprezintă un set de reguli și convenții care definesc modul în care datele sunt transmise între dispozitive într-o rețea [5]. Fără aceste reguli standardizate, dispozitivele produse de diferiți producători nu ar putea comunica între ele. În cadrul stivei TCP/IP, protocoalele sunt organizate ierarhic, fiecare nivel adăugând funcționalități specifice procesului de transmisie.

2.2.1 IP, TCP, UDP

Protocolul IP (Internet Protocol) operează la nivelul rețea și are responsabilitatea principală de a adresa și ruta pachetele de date între sursa și destinația lor, indiferent de rețelele intermediare traversate [7]. IP este un protocol fără conexiune și fără garanții de livrare: nu asigură că pachetele ajung la destinație, nu garantează ordinea de sosire și nu detectează erorile de transmisie. Aceste responsabilități revin protocoalelor de la niveluri superioare.

Versiunea larg utilizată astăzi este IPv4, care folosește adrese pe 32 de biți, permițând teoretic

aproximativ 4,3 miliarde de adrese unice [6]. Din cauza epuizării spațiului de adresare IPv4, a fost dezvoltat IPv6, care utilizează adrese pe 128 de biți, oferind un spațiu de adresare practic nelimitat. Headerul unui pachet IPv4 conține câmpuri esențiale precum adresa sursă, adresa destinație, Time-To-Live (TTL) și protocolul utilizat la nivelul superior.

Protocolul TCP (Transmission Control Protocol) operează la nivelul transport și oferă un serviciu de comunicație orientat pe conexiune, fiabil și ordonat [8]. Înainte de transmiterea oricăror date, TCP stabilește o conexiune printr-un mecanism numit three-way handshake: clientul trimite un segment SYN, serverul răspunde cu SYN-ACK, iar clientul confirmă cu ACK. Abia după finalizarea acestui schimb pot fi transmise date aplicative.

TCP garantează livrarea corectă și în ordine a datelor prin mecanisme de confirmare (acknowledgement), retransmisie în caz de pierdere și control al fluxului. Aceste caracteristici îl fac protocolul preferat pentru aplicații în care integritatea datelor primează: navigare web (HTTP/HTTPS), transfer de fișiere (FTP), email (SMTP) [5].

Protocolul UDP (User Datagram Protocol) este, prin contrast, un protocol fără conexiune și fără garanții de livrare [9]. UDP nu stabilește o conexiune înainte de transmisie, nu confirmă recepția pachetelor și nu retransmite datele pierdute. Această simplitate îi conferă o latență semnificativ mai mică față de TCP, ceea ce îl face adecvat pentru aplicații în care viteza primează față de fiabilitate: streaming audio/video, jocuri online, interogări DNS. În contextul sistemelor IDS, cunoașterea comportamentului UDP este relevantă deoarece numeroase atacuri de tip DoS/DDoS utilizează inundații de pachete UDP.

O comparație sintetică între cele două protocoale de transport este prezentată în Tabelul 2.1.

Caracteristică	TCP	UDP
Tip conexiune	Orientat pe conexiune	Fără conexiune
Garanție livrare	Da	Nu
Ordine pachete	Garantată	Negarantată
Viteză	Mai lentă	Mai rapidă
Utilizare tipică	HTTP, FTP, SMTP	DNS, streaming, VoIP

Tabela 2.1: Comparație TCP vs UDP [5, 6]

2.2.2 Porturi și sesiuni

Un port este un număr pe 16 biți (0–65535) asociat unui proces sau serviciu care rulează pe un dispozitiv, permițând sistemului de operare să direcționeze pachetele primite către aplicația corectă [6]. Combinația dintre o adresă IP și un număr de port formează un socket, care identifică în mod unic un capăt al unei conexiuni de rețea.

Porturile sunt împărțite în trei categorii [5]:

- Porturi bine cunoscute (0–1023) - rezervate serviciilor standardizate: portul 80 pentru HTTP, portul 443 pentru HTTPS, portul 22 pentru SSH, portul 21 pentru FTP;

- Porturi înregistrate (1024–49151) - alocate de IANA aplicațiilor specifice, dar fără caracter exclusiv;
- Porturi dinamice/efemere (49152–65535) - alocate temporar de sistemul de operare pentru conexiunile inițiate de client.

O sesiune TCP reprezintă întreaga durată de viață a unei conexiuni, de la stabilire până la închidere. Stabilirea sesiunii are loc prin three-way handshake descris anterior, iar închiderea se realizează printr-un schimb de patru pași folosind flag-urile FIN și ACK. Fiecare sesiune activă este identificată de un quadruplet unic: (IP sursă, port sursă, IP destinație, port destinație). Această reprezentare stă la baza conceptului de flux de rețea (network flow), esențial în detectarea anomaliilor descrisă în capitolele următoare [5].

2.3 Tipuri de trafic în rețea

Traficul de rețea reprezintă totalitatea datelor care circulă între dispozitivele conectate într-o rețea de calculatoare [5]. Analiza acestuia este esențială atât pentru monitorizarea și optimizarea performanței rețelei, cât și pentru detectarea și prevenirea activităților suspecte sau malițioase. În domeniul securității informatice, clasificarea traficului în categorii distincte permite dezvoltarea unor mecanisme de detecție a intruziunilor mai eficiente, adaptate la specificul fiecărui tip de trafic. Din punct de vedere al comportamentului, traficul de rețea poate fi împărțit în două categorii principale: trafic normal și trafic anormal.

2.3.1 Trafic normal

Traficul normal reprezintă fluxul de date generat de activități legitime ale utilizatorilor și aplicațiilor, care respectă protocoalele și regulile de comunicare stabilite. Acesta include operațiuni precum accesarea paginilor web, transferul de fișiere, utilizarea serviciilor de email, streaming etc. sau comunicarea prin aplicații de tip client-server.

Acest tip de trafic prezintă, în general, următoarele caracteristici:

- respectarea protocoalelor standard de comunicație (HTTP, FTP, SMTP, DNS etc.);
- distribuție relativ constantă a volumului de date și a numărului de conexiuni pe unitatea de timp;
- număr moderat și previzibil de conexiuni simultane;
- comportament previzibil al fluxurilor de date, fără variații bruște sau anomalii în tiparele de trafic.

În sistemele moderne de detecție a intruziunilor bazate pe Machine Learning, traficul normal este utilizat pentru definirea și antrenarea modelelor de detecție a anomaliilor, reprezentând baza de referință pentru identificarea comportamentelor suspecte sau malițioase în rețea [6].

2.3.2 Trafic anormal

Traficul anormal, cunoscut și sub denumirea de trafic malițios sau suspect, reprezintă orice tip de activitate în rețea care se abate semnificativ de la comportamentul obișnuit și care poate indica prezența unei amenințări cibernetice, a unui atac informatic sau a unei utilizări neautorizate a resurselor. Acesta poate apărea sub diverse forme, printre care:

- atacuri de tip DoS/DDoS;
- scanări de porturi;
- tentative de autentificare de tip brute-force;
- trafic generat de programe malițioase (malware);

Spre deosebire de traficul normal, traficul anormal prezintă adesea următoarele caracteristici:

- variații bruște și neobișnuite ale volumului de date;
- creșteri neobișnuite ale numărului de conexiuni simultane;
- distribuții suspecte ale adreselor IP și porturilor implicate în comunicare;
- modele repetitive neobișnuite.

Detectarea acestui tip de trafic reprezintă o provocare majoră pentru sistemele de securitate, în special în cazul atacurilor noi sau necunoscute. În timp ce sistemele clasice bazate pe semnături pot identifica doar amenințări deja cunoscute, metodele bazate pe Machine Learning au potențialul de a detecta anomalii prin analizarea deviațiilor de la comportamentul normal al rețelei [1, 11].

2.4 Fluxuri de rețea (network flows)

Conceptul de flux de rețea reprezintă unitatea fundamentală de analiză utilizată în sistemele moderne de detecție a intruziunilor [5]. Spre deosebire de abordarea la nivel de pachet individual, analiza la nivel de flux permite extragerea unor caracteristici statistice și comportamentale care descriu mai bine natura unei comunicații, facilitând astfel identificarea tiparelor anormale.

2.4.1 Definiție

Un flux de rețea (network flow) este definit ca o secvență de pachete cu aceleași valori pentru un set de câmpuri cheie, transmise într-un interval de timp delimitat [13]. În practica sistemelor IDS, identificarea unui flux se bazează pe un 5-tuplu:

- Adresa IP sursă

- Adresa IP destinație
- Portul sursă
- Portul destinație
- Protocolul de transport (TCP, UDP etc.)

Toate pachetele care au aceleași valori pentru aceste cinci câmpuri aparțin aceluiași flux. Fluxurile pot fi unidirecționale -considerând doar pachetele dintr-o singură direcție - sau bidirecționale - agregând pachetele din ambele sensuri ale comunicației. Dataset-ul CICIDS2017, utilizat în această lucrare, folosește fluxuri bidirecționale generate cu instrumentul CICFlowMeter [12].

Un flux se consideră încheiat în unul din următoarele cazuri [13]:

- La recepționarea unui pachet TCP cu flag-ul FIN sau RST (închiderea normală sau forțată a conexiunii);
- La expirarea unui timeout de inactivitate (de obicei 60 de secunde pentru fluxuri active și 15 secunde pentru fluxuri inactive);
- La atingerea unei durate maxime configurabile a fluxului.

Această reprezentare la nivel de flux reduce semnificativ volumul de date față de analiza pachet-cu-pachet, păstrând în același timp informații comportamentale suficiente pentru detectarea anomaliilor. Un trafic de tip DoS, de exemplu, se manifestă prin fluxuri cu un număr extrem de mare de pachete pe secundă, în timp ce un atac de tip port scanning generează numeroase fluxuri de durată foarte scurtă [12].

2.4.2 Caracteristici (features)

Din fiecare flux de rețea pot fi extrase caracteristici numerice care descriu comportamentul comunicației [12]. Aceste caracteristici formează vectorul de intrare al modelului de Machine Learning. Alegerea și calitatea acestor caracteristici influențează direct performanța sistemului de detecție.

Caracteristicile utilizate în această lucrare sunt organizate în mai multe categorii:

Caracteristici temporale:

- Duration - durata totală a fluxului, în microsecunde;
- Flow IAT Mean / Max / Min / Std - statistici privind timpii de inter-sosire (Inter-Arrival Time) între pachetele fluxului;
- Fwd IAT / Bwd IAT - timpii de inter-sosire separați pentru direcția forward și backward.

Caracteristici volumetrice:

- Total Fwd Packets / Bwd Packets - numărul total de pachete în fiecare direcție;
- Total Length of Fwd / Bwd Packets - numărul total de octeți transportați;
- Flow Bytes/s și Flow Packets/s - rata de transfer exprimată în octeți, respectiv pachete per secundă.

Caracteristici statistice ale lungimii pachetelor:

- Packet Length Mean / Max / Min / Std / Variance -statistici descriptive privind dimensiunea pachetelor din flux.

Caracteristici bazate pe flag-uri TCP:

- Numărul de pachete cu flag-urile SYN, ACK, FIN, RST, PSH, URG active -relevante pentru identificarea tipului de conexiune și a anomaliilor de protocol.

Aceste caracteristici sunt extrase automat de instrumentul CICFlowMeter din capturile de trafic în format PCAP și exportate într-un fișier CSV, care constituie intrarea sistemului de detecție propus în această lucrare [12].

Capitolul 3

Securitate rețea și sisteme IDS

3.1 Tipuri de atacuri informatice

Securitatea rețelelor de calculatoare este amenințată de o varietate de atacuri informatice, fiecare cu mecanisme și obiective distincte. Înțelegerea tipologiei acestor atacuri este esențială pentru proiectarea unor sisteme de detecție eficiente, deoarece fiecare tip de atac generează tipare specifice în traficul de rețea [1]. În această secțiune sunt prezentate principalele categorii de atacuri relevante pentru sistemul IDS propus în această lucrare.

3.1.1 DoS / DDoS

Atacul de tip DoS (Denial of Service) are ca obiectiv perturbarea sau întreruperea completă a disponibilității unui serviciu, server sau rețea, prin copleșirea acestuia cu un volum excesiv de cereri sau trafic [1]. Spre deosebire de atacurile care vizează furtul de date, atacul DoS urmărește exclusiv indisponibilizarea resursei țintă pentru utilizatorii legitimi.

Atacul de tip DDoS (Distributed Denial of Service) reprezintă o formă distribuită a atacului DoS, în care traficul malițios provine simultan de la un număr mare de sisteme compromise, denumite colectiv botnet [2]. Coordonarea unui botnet permite generarea unui volum de trafic imposibil de blocat prin filtrarea unei singure surse, ceea ce face atacurile DDoS semnificativ mai dificil de contracarat față de varianta clasică DoS.

Din perspectiva tipului de trafic generat, atacurile DoS/DDoS pot fi clasificate în trei categorii principale [1]:

- Atacuri volumetrice - inundarea lățimii de bandă cu pachete UDP sau ICMP în volume foarte mari (UDP Flood, ICMP Flood);
- Atacuri de protocol - exploatarea vulnerabilităților din stiva TCP/IP, cel mai cunoscut exemplu fiind SYN Flood, care epuizează resursele serverului prin inițierea unui număr mare de conexiuni TCP nefinalizate;

- Atacuri la nivelul aplicației - solicitări aparent legitime care supraîncarcă resursele țintei (HTTP Flood).

În traficul de rețea, aceste atacuri se caracterizează prin creșteri bruște ale numărului de pachete pe secundă și un raport asimetric între traficul de intrare și cel de ieșire al țintei [12].

3.1.2 Brute force

Atacul de tip brute force constă în încercarea sistematică a unui număr mare de combinații de credențiale (nume de utilizator și parolă) până la identificarea uneia valide, cu scopul obținerii accesului neautorizat la un sistem sau serviciu [1]. Acest tip de atac nu exploatează vulnerabilități tehnice ale sistemului, ci se bazează pe forța de calcul și pe alegerea unor parole slabe de către utilizatori.

Forma cea mai simplă presupune testarea exhaustivă a tuturor combinațiilor posibile de caractere, fără nicio informație prealabilă. O variantă mai eficientă este atacul de tip dicționar (dictionary attack), care utilizează o listă predefinită de parole frecvent folosite sau obținute din scurgeri anterioare de date. O a treia variantă, credential stuffing, reutilizează perechi de credențiale compromise din breșe de securitate anterioare pentru a testa accesul pe alte platforme [2]. Serviciile cel mai frecvent vizate sunt SSH, FTP, RDP și formularele de autentificare web [1]. În traficul de rețea, atacul se manifestă printr-un număr mare de conexiuni eșuate într-un interval scurt de timp, provenind de la aceeași sursă, către același port destinație.

3.1.3 Port scanning

Scanarea porturilor (port scanning) este o tehnică de recunoaștere utilizată de atacatori pentru a identifica porturile deschise și serviciile disponibile pe un sistem țintă, în vederea planificării unui atac ulterior [1]. Deși nu reprezintă în sine un atac direct, scanarea porturilor precede adesea alte tipuri de atacuri și constituie un semnal important pentru sistemele IDS.

Cea mai utilizată tehnică este TCP SYN Scan (half-open scan), care trimite pachete SYN și analizează răspunsurile fără a finaliza conexiunea TCP - un port deschis răspunde cu SYN-ACK, unul închis cu RST. Aceasta este preferată datorită discreției relative. O alternativă mai simplă, TCP Connect Scan, stabilește conexiuni complete și nu necesită privilegii speciale, dar este mai ușor de detectat. Pentru serviciile UDP, UDP Scan detectează porturile deschise prin absența unui răspuns ICMP de tip port unreachable [2]. În traficul de rețea, scanarea porturilor generează un număr mare de conexiuni de scurtă durată sau pachete respinse (RST), provenind de la o singură sursă și adresate unui număr mare de porturi ale aceleiași destinații, într-un interval redus de timp [12].

3.1.4 Malware

Termenul malware (malicious software) desemnează orice program sau cod conceput cu intenție malițioasă, care afectează funcționarea sistemelor informatice, fură date sau facilitează accesul neautorizat la resurse [1]. Din perspectiva traficului de rețea, malware-ul este relevant prin comunicațiile pe care le generează, adesea detectabile ca anomalii de comportament.

Principalele categorii de malware cu impact semnificativ asupra traficului de rețea sunt [2]:

- Botnet - rețea de dispozitive infectate controlate printr-un server de comandă și control (C&C); traficul periodic cu acest server este detectabil prin analiza fluxurilor;
- Trojan - program aparent legitim care execută funcții malițioase în fundal, precum exfiltrarea datelor sau deschiderea unui backdoor;
- Ransomware - cifrează fișierele victimei și generează, în faza de propagare, trafic de scanare similar port scanning-ului;
- Spyware - colectează și transmite informații despre utilizator, generând fluxuri suspecte de date către servere externe.

Detectarea acestor amenințări prin analiza traficului se bazează pe identificarea conexiunilor periodice către adrese IP externe necunoscute, a transferurilor mari de date în afara orelor de program sau a utilizării unor protocoale neobișnuite pe porturi standard [1].

3.2 Sisteme de detecție a intruziunilor (IDS)

Sistemele de detecție a intruziunilor reprezintă o componentă esențială a infrastructurii de securitate a rețelelor moderne. Apărute ca răspuns la limitările mecanismelor de securitate pasivă, precum firewall-urile și listele de control al accesului, aceste sisteme adaugă un nivel activ de monitorizare capabil să identifice activitățile suspecte în timp real [1].

3.2.1 Definiție

Un sistem de detecție a intruziunilor (IDS - Intrusion Detection System) este definit de NIST ca un software care automatizează procesul de monitorizare a evenimentelor produse într-un sistem informatic sau rețea de calculatoare, cu scopul de a detecta semne ale unor posibile incidente de securitate [2]. Prin incident de securitate se înțelege orice tentativă de compromitere a confidențialității, integrității sau disponibilității informațiilor, indiferent dacă aceasta provine din exterior sau din interiorul organizației.

Un sistem IDS este alcătuit, în general, din trei componente principale: un modul de colectare a datelor (senzori sau agenți), un modul de analiză care procesează datele colectate și identifică tiparele suspecte, și o interfață de management prin care administratorii primesc alertele generate

și pot configura comportamentul sistemului [2]. Datele analizate pot include pachete de rețea, jurnale de sistem, apeluri de sistem sau evenimente de autentificare, în funcție de tipul de IDS utilizat.

Spre deosebire de sistemele de prevenire a intruziunilor (IPS - Intrusion Prevention System), care pot interveni activ pentru a bloca sau modifica traficul suspect [1], un IDS este un sistem pasiv care observă și raportează, fără a interveni în fluxul de date.

3.2.2 Rol

Rolul principal al unui sistem IDS este de a oferi vizibilitate asupra activităților din rețea sau de pe sistemele monitorizate, permițând administratorilor să identifice și să răspundă rapid la amenințările de securitate [2]. Fără un astfel de mecanism, atacurile pot rămâne nedetectate pentru perioade îndelungate, amplificând considerabil impactul acestora.

Din perspectivă operațională, un sistem IDS îndeplinește mai multe funcții complementare [1]. Monitorizează în timp real traficul de rețea sau activitatea sistemului, comparând-o cu modele de comportament normal sau cu semnături de atacuri cunoscute. La detectarea unei anomalii, generează alerte care sunt transmise administratorilor sau centralizate într-un sistem SIEM (Security Information and Event Management). De asemenea, înregistrează evenimentele detectate în jurnale detaliate, utile atât pentru investigarea incidentelor, cât și pentru demonstrarea conformității cu reglementările de securitate.

În contextul acestei lucrări, rolul sistemului IDS propus este de a analiza fluxurile de rețea capturate în timp real și de a clasifica fiecare flux ca trafic normal sau anormal, folosind un model de Machine Learning antrenat pe dataset-ul CICIDS2017. Spre deosebire de sistemele clasice, care se bazează pe reguli sau semnături predefinite, abordarea propusă urmărește detectarea atacurilor necunoscute prin identificarea deviațiilor de la comportamentul normal al rețelei [12].

3.3 Clasificarea IDS

Sistemele de detecție a intruziunilor pot fi clasificate după metoda utilizată pentru identificarea activității suspecte [2]. Distincția fundamentală se face între sistemele care recunosc amenințările cunoscute pe baza unor șabloane predefinite și cele care detectează abateri de la comportamentul normal al rețelei. Această clasificare este esențială pentru înțelegerea limitărilor și a avantajelor fiecărei abordări, prezentate în subsecțiunile următoare.

3.3.1 Signature-based

Detectarea bazată pe semnături reprezintă cea mai răspândită abordare în implementările comerciale actuale [1]. Un sistem de acest tip menține o bază de date de semnături - șabloane sau reguli care descriu caracteristicile cunoscute ale atacurilor - și compară fiecare eveniment

sau pachet inspectat cu această bază. Dacă se găsește o corespondență, sistemul generează o alertă.

Avantajul principal al acestei abordări constă în precizia ridicată în identificarea atacurilor cunoscute și în rata scăzută a fals-pozitivelor, datorată specificității semnăturilor [2]. Procesarea este eficientă, iar rezultatele sunt ușor de interpretat de către administratorii de rețea, fiecare alertă putând fi corelată direct cu o semnătură specifică.

Limitarea critică este incapacitatea de a detecta atacurile noi sau variantele modificate ale atacurilor cunoscute, deoarece acestea nu posedă o semnătură în baza de date [1]. Mai mult, menținerea bazei de semnături necesită actualizări continue, ceea ce implică un efort operațional semnificativ și o fereastră de vulnerabilitate între apariția unui atac nou și publicarea semnăturii corespunzătoare.

3.3.2 Anomaly-based

Detectarea bazată pe anomalii operează printr-un mecanism diferit față de abordarea bazată pe semnături [1, 2]. În loc să recunoască atacuri specifice, sistemul construiește un model al comportamentului normal al rețelei și identifică deviațiile semnificative față de acesta. Orice flux sau eveniment care se abate substanțial de la profilul de normalitate este clasificat drept potențial malițios.

Principalul avantaj al acestei abordări este capacitatea de a detecta atacuri necunoscute, inclusiv amenințări de tip zero-day, fără a necesita actualizarea manuală a bazei de semnături [2]. Modelul de normalitate este construit în faza de antrenare, pe baza traficului legitim, și poate surprinde tipare comportamentale complexe care ar fi dificil de descris prin reguli explicite.

Contrapartea este o rată mai ridicată a fals-pozitivelor, deoarece comportamentul legitim neobișnuit poate fi interpretat ca o anomalie [1]. Performanța sistemului depinde în mod direct de calitatea și reprezentativitatea datelor de antrenare. Sistemul propus în această lucrare aparține acestei categorii, utilizând modele de Machine Learning antrenate pe fluxuri de trafic din dataset-ul CICIDS2017 [12].

3.3.3 Rule-based

Detectarea bazată pe reguli combină elemente din cele două abordări anterioare [2]. În loc să folosească semnături ale atacurilor sau profiluri statistice de comportament normal, acest tip de sistem utilizează seturi de reguli care definesc comportamentul legitim al protocoalelor și serviciilor, derivate de obicei din specificațiile formale ale acestora. Atunci când comportamentul observat încalcă una dintre reguli, este generată o alertă.

Avantajul față de detectarea pur bazată pe anomalii este că regulile sunt explicabile și verificabile, ceea ce reduce rata fals-pozitivelor și simplifică auditarea sistemului [1]. Dezavantajul constă în efortul necesar definirii și menținerii unui set complet de reguli, mai ales pentru

protocoale complexe sau aplicații cu comportament dinamic. În practică, abordările moderne combină toate cele trei metode pentru a compensa limitările individuale ale fiecăreia.

3.4 Tipuri de IDS

Pe lângă clasificarea după metoda de detecție, sistemele IDS pot fi diferențiate și după locul de implementare și sfera de monitorizare [1, 2]. Principalele tipuri din această perspectivă sunt sistemele bazate pe rețea (NIDS) și cele bazate pe gazdă (HIDS), fiecare acoperind scenarii de utilizare distincte și complementare.

3.4.1 NIDS (Network-based)

Un sistem de detecție a intruziunilor bazat pe rețea (NIDS - Network-based Intrusion Detection System) monitorizează traficul de rețea la nivel de infrastructură, analizând pachetele sau fluxurile care circulă prin segmentele de rețea supravegheate [2]. Senzorul NIDS este plasat la puncte strategice ale rețelei - la perimetru, în zona demilitarizată (DMZ) sau pe segmentele interne critice - și interceptează pasiv traficul fără a perturba comunicațiile.

Avantajul principal al unui NIDS constă în capacitatea de a monitoriza simultan toate dispozitivele dintr-un segment de rețea, fără a necesita instalarea de software pe fiecare gazdă [14]. Un singur senzor poate asigura vizibilitate asupra întregului trafic al unui segment, ceea ce îl face eficient din punct de vedere al costurilor de administrare. De asemenea, NIDS este transparent pentru utilizatorii și aplicațiile monitorizate.

Limitarea principală apare în contextul traficului criptat: un senzor NIDS plasat în afara terminalului de criptare nu poate inspecta conținutul pachetelor protejate prin TLS/SSL [1]. În rețelele moderne, unde criptarea end-to-end este generalizată, această limitare reduce eficacitatea NIDS în detectarea atacurilor la nivel de aplicație. Sistemul propus în această lucrare adoptă o abordare de tip NIDS, analizând fluxurile de trafic capturate la nivelul rețelei prin CICFlowMeter [12].

3.4.2 HIDS (Host-based)

Un sistem de detecție a intruziunilor bazat pe gazdă (HIDS - Host-based Intrusion Detection System) funcționează la nivelul unui singur dispozitiv, monitorizând activitățile locale ale acestuia [1, 2]. Sursele de date analizate includ apelurile de sistem, jurnalele de aplicații, modificările fișierelor de sistem, activitatea proceselor și evenimentele de autentificare.

Avantajul distinctiv al HIDS este capacitatea de a detecta atacuri care nu generează trafic de rețea vizibil din exterior, cum ar fi escaladarea privilegiilor, modificarea fișierelor critice sau activitatea malware care operează exclusiv în memoria gazdei [14]. În plus, HIDS poate

inspecta traficul criptat după ce acesta a fost decriptat la nivelul aplicației locale, depășind astfel limitarea fundamentală a NIDS.

Dezavantajul principal constă în necesitatea instalării și gestionării unui agent pe fiecare gazdă monitorizată, ceea ce implică consum de resurse locale și un efort de administrare proporțional cu numărul de sisteme din rețea [2]. În mediile enterprise cu sute sau mii de dispozitive, această cerință poate deveni un obstacol semnificativ. În practică, arhitecturile moderne de securitate combină NIDS și HIDS pentru a obține o acoperire completă atât a activității de rețea, cât și a celei de pe fiecare gazdă.

3.5 Limitările sistemelor clasice

Sistemele IDS clasice, deși fundamentale pentru securitatea rețelelor, prezintă limitări structurale care le afectează eficiența în fața amenințărilor cibernetice moderne [1, 14]. Aceste limitări nu sunt accidentale, ci decurg din înseși principiile de funcționare ale abordărilor tradiționale, bazate pe semnături și reguli statice. Înțelegerea acestor constrângeri este esențială pentru a motiva adoptarea tehnicilor bazate pe machine learning.

3.5.1 Imposibilitatea detectării atacurilor noi

Sistemele IDS bazate pe semnături sunt în mod inerent incapabile să detecteze atacuri care nu au mai fost observate anterior [14]. Această limitare fundamentală derivă din dependența lor de baze de date care trebuie să conțină o reprezentare prealabilă a unui atac pentru ca acesta să poată fi identificat. Atacurile de tip zero-day - exploatări ale vulnerabilităților descoperite și utilizate înainte ca producătorii de software să fi emis un patch - reprezintă tocmai scenariul în care sistemele bazate pe semnături eșuează cel mai categoric.

Fenomenul este agravat de viteza cu care noile tehnici de atac sunt dezvoltate și distribuite. Intervalul dintre descoperirea unei vulnerabilități și exploatarea sa activă poate fi de câteva ore sau chiar minute, un interval mult prea scurt pentru ca o semnătură să fie creată, testată și distribuită sistemelor de detecție [1]. Această realitate a determinat industria să caute abordări complementare, în special metode bazate pe detecția anomaliilor.

Sistemele bazate pe anomalii au potențialul de a identifica atacuri noi prin detectarea deviațiilor față de comportamentul normal al rețelei, fără a depinde de cunoașterea prealabilă a atacului [2]. De aceea, tehnicile de machine learning sunt preferate în sistemele moderne de detecție a intruziunilor, inclusiv în abordarea prezentată în această lucrare.

3.5.2 Dependența de semnături

Semnăturile reprezintă coloana vertebrală a sistemelor IDS clasice, dar această dependență introduce vulnerabilități structurale semnificative [1]. Pentru a rămâne eficiente, bazele de

date de semnături necesită actualizări continue, ceea ce implică o infrastructură permanentă de cercetare, testare și distribuție. Organizațiile care nu mențin aceste baze de date la zi se expun riscului de a nu detecta amenințări recent apărute.

Un aspect și mai problematic este capacitatea atacatorilor de a eluda deliberat sistemele bazate pe semnături prin tehnici de obfuscare și polimorfism [14]. Malware-ul polimorfic, de exemplu, își modifică codul la fiecare instanțiere, generând variante cu semnături distincte față de exemplarele cunoscute, fără a-și altera funcționalitatea malițioasă. Tehnicile de fragmentare a pachetelor, codarea alternativă a payload-ului și utilizarea protocoalelor nestandard sunt metode prin care un atacator poate traversa un IDS bazat pe semnături fără a declanșa alarme.

Scalabilitatea reprezintă o altă dimensiune a acestei dependențe: pe măsură ce numărul amenințărilor cunoscute crește exponențial, bazele de date de semnături devin din ce în ce mai mari, iar procesul de comparare a traficului cu un număr tot mai mare de semnături introduce latențe și consumă resurse computaționale semnificative [2].

3.5.3 Problema fals-pozitivelor

Rata ridicată de fals-pozitive reprezintă una dintre cele mai critice limitări practice ale sistemelor IDS [14]. Un fals-pozitiv apare atunci când sistemul clasifică activitate legitimă drept potențial malițioasă, generând o alertă care nu corespunde unui incident real de securitate. Impactul acumulat al unui volum mare de alerte false este fenomenul de alert fatigue, în care analiștii de securitate devin desensibilizați la alarme și pot rata amenințări reale tocmai din cauza saturației cu notificări false.

Sistemele bazate pe anomalii sunt deosebit de predispuse la fals-pozitive atunci când comportamentul normal al rețelei variază semnificativ, de exemplu în perioadele de actualizări software sau evenimente care generează vârfuri de trafic neobișnuite [1]. Calibrarea pragurilor de alertare devine un compromis delicat: un prag prea sensibil generează numeroase fals-pozitive, în timp ce un prag prea permisiv riscă să producă fals-negative, adică amenințări reale care trec nedetectate.

Costul operațional al investigării manuale a alertelor false este semnificativ, mai ales în organizațiile mari unde volumul traficului poate genera sute sau mii de alerte pe zi [2]. Reducerea ratei de fals-pozitive fără a compromite rata de detectare a amenințărilor reale este unul dintre obiectivele centrale ale cercetărilor actuale în domeniu, inclusiv prin utilizarea algoritmilor de machine learning capabili să învețe reprezentări mai nuanțate ale comportamentului normal al rețelei.

Capitolul 4

Machine Learning pentru detecția anomaliilor

4.1 Introducere în Machine Learning

Machine learning este un subdomeniu al inteligenței artificiale care studiază algoritmii și modelele statistice ce permit sistemelor informatice să își îmbunătățească performanța asupra unor sarcini specifice prin experiență, fără a fi explicit programate pentru fiecare situație [15]. Disciplina s-a dezvoltat la intersecția statisticii, teoriei probabilităților și informaticii teoretice, devenind astăzi una dintre cele mai active arii de cercetare aplicată. În contextul detecției intruziunilor, machine learning furnizează cadrul teoretic și practic pentru construirea de sisteme capabile să identifice tipare de atac direct din date de trafic, depășind limitările fundamentale ale abordărilor bazate pe semnături.

Principiul unificator al tuturor metodelor de machine learning este că modelul extrage relații și structuri din date, generalizând observațiile din setul de antrenare la cazuri noi, nevăzute anterior [16]. Procesul presupune alegerea unei reprezentări a datelor (vectori de caracteristici), definirea unui criteriu de optimizare (funcție de pierdere sau scor de anomalie) și un algoritm de căutare în spațiul parametrilor modelului. Calitatea generalizării depinde de cantitatea și calitatea datelor disponibile, de adecvarea reprezentării și de regularizarea aplicată pentru a preveni supraajustarea.

4.1.1 Supervised learning

Învățarea supervizată este paradigma de machine learning în care modelul este antrenat pe un set de date etichetat, compus din perechi (intrare, etichetă corectă), cu scopul de a învăța o funcție de mapare de la intrări la ieșiri [15, 16]. Algoritmul minimizează o funcție de pierdere pe setul de antrenare, ajustând parametrii interni astfel încât predicțiile să corespundă cât mai bine etichetelor cunoscute. În contextul detectiei intruziunilor, învățarea supervizată se aplică atunci când sunt disponibile seturi de date etichetate cu fluxuri de trafic normal și malițios, cum

este CICIDS2017 utilizat în această lucrare.

Algoritmii frecvent utilizați în cadrul acestui paradigm includ arborii de decizie, pădurile aleatoare, mașinile cu vectori suport, regresia logistică și rețelele neuronale [16]. Fiecare algoritm operează pe ipoteze diferite privind distribuția datelor și prezintă compromisuri distincte între interpretabilitate, complexitate computațională și capacitate de generalizare. Metodele de ansamblu, precum Random Forest, s-au dovedit deosebit de eficiente pentru clasificarea traficului de rețea datorită robusteții lor la zgomot și capacității de a trata spații de caracteristici de dimensiune ridicată.

Principalul avantaj al învățării supervizate constă în acuratețea ridicată obținută atunci când datele etichetate sunt suficiente și reprezentative [11, 12]. Obținerea unor astfel de date în domeniul securității presupune însă adnotarea manuală a fluxurilor de trafic de către experți, proces costisitor în timp și resurse. Mai mult, modelele supervizate antrenate pe tipuri de atac cunoscute pot generaliza slab la variante noi, ceea ce limitează capacitatea lor de a detecta amenințări zero-day.

4.1.2 Unsupervised learning

Învățarea nesupervizată operează exclusiv pe date neetichetate, urmărind să descopere structuri latente sau tipare statistice fără categorii predefinite [16]. Algoritmul nu primește feedback explicit despre corectitudinea reprezentărilor pe care le construiește, ci organizează datele pe baza proprietăților lor intrinseci, cum ar fi densitatea locală, distanțele sau corelațiile între caracteristici. În detecția intruziunilor, metodele nesupervizate modelează distribuția traficului normal și semnalează devierile față de aceasta ca potențiale anomalii.

Principalele categorii de algoritmi nesupervizați includ metodele de grupare (k-means, DBSCAN, grupare ierarhică) și tehnicile de reducere a dimensionalității (PCA, autoencoare) [16]. Isolation Forest, unul dintre algoritmii evaluați în această lucrare, este o metodă nesupervizată de detecție a anomaliilor care izolează instanțele anormale prin partiționarea aleatorie repetată a spațiului de caracteristici, exploatând faptul că anomaliile sunt puține la număr și structural distincte față de punctele normale.

Avantajul esențial al metodelor nesupervizate în aplicații de securitate este independența față de date etichetate, ceea ce le face aplicabile în medii în care exemplele de atac sunt rare sau indisponibile [11]. Contra-partida este o rată mai mare de fals-pozitive față de metodele supervizate, deoarece tipare de trafic legitim, dar neobișnuite - generate de actualizări software sau vârfuri de activitate - pot fi detectate eronat ca anomalii.

4.1.3 Semi-supervised learning

Învățarea semi-supervizată combină un set mic de date etichetate cu un volum mult mai mare de date neetichetate în procesul de antrenare, plasându-se între paradigmele supervizată și nesupervizată [17]. Ipoteza de bază este că datele neetichetate, deși lipsite de etichete

explicite, conțin informații despre structura spațiului de intrare care pot îmbunătăți generalizarea modelului. Cazul detectiei intruziunilor se potrivește acestei paradigme: traficul normal este abundent și ușor de colectat, dar etichetarea atacurilor necesită expertiză specializată.

În abordările semi-supervizate pentru IDS, modelul este antrenat tipic pe un set format predominant din trafic normal neetichetat, la care se adaugă un subset restrâns de exemple de atac etichetate [17]. Tehnici precum propagarea etichetelor, auto-antrenarea și modelele generative sunt folosite pentru a exploata informațiile structurale din datele neetichetate în scopul calibrării mai precise a frontierei de decizie. Modelul învață astfel o reprezentare mai bogată a spațiului de date decât ar fi posibil folosind exclusiv exemplele etichetate.

One-Class SVM, al doilea algoritm evaluat în această lucrare, poate fi interpretat ca o metodă semi-supervizată atunci când este antrenat exclusiv pe trafic normal, construind o frontieră compactă în jurul distribuției traficului benign și clasificând orice deviație semnificativă ca anomalie. Abordarea este utilă în medii operaționale reale, unde etichetarea exhaustivă a atacurilor este imposibilă, iar sistemul trebuie să detecteze amenințări noi pornind de la cunoașterea comportamentului normal.

4.2 Detecția anomaliilor

Detecția anomaliilor este un subdomeniu al machine learning și analizei de date care abordează problema identificării instanțelor, tiparelor sau evenimentelor ce se abat semnificativ de la comportamentul normal așteptat [18]. Conceptul are rădăcini în statistica clasică, unde metodele de detecție a valorilor aberante (outliers) au fost studiate pentru identificarea observațiilor inconsistente cu un model statistic. În contextul securității rețelelor, detecția anomaliilor oferă un cadru principial pentru identificarea activității malițioase fără cunoașterea prealabilă a semnăturilor specifice de atac.

4.2.1 Definiție

O anomalie, denumită și outlier în literatura de specialitate, este o observație care se abate atât de mult de la celelalte observații încât ridică suspiciunea că a fost generată de un mecanism diferit [18]. Formal, dat un set de date de antrenare reprezentând comportament normal, sarcina de detecție a anomaliilor constă în asignarea fiecărei noi instanțe a unui scor care reflectă gradul de deviere față de distribuția normală, instanțele cu scor peste un prag prestabilit fiind clasificate ca anomalii.

Din perspectivă model-based, procesul se desfășoară în două faze: în faza de antrenare se construiește un model al comportamentului normal, iar în faza de inferență noile observații sunt comparate cu acest model [16]. Alegerea pragului de decizie controlează direct compromisul dintre sensibilitate (rata de detecție a atacurilor reale) și specificitate (rata de fals-pozitive), un parametru critic pentru aplicabilitatea practică a sistemului.

În contextul IDS bazat pe fluxuri de rețea, comportamentul normal este definit prin proprietăți statistice ale fluxurilor, precum rata de pachete, distribuția volumului de octeți, durata conexiunilor și proporția pachetelor cu diferite flag-uri TCP [14]. O anomalie la nivel de rețea poate corespunde unui volum excesiv de cereri de conexiune (atac DoS), unor tipare neobișnuite de accesare a porturilor (scanare) sau unui volum anormal de date trimise în exterior (exfiltrare de date).

4.2.2 Tipuri de anomalii

Literatura de specialitate distinge trei categorii principale de anomalii în funcție de natura și contextul abaterilor observate [18]. Prima categorie, anomaliile punctuale (point anomalies), se referă la instanțe individuale care sunt anormale față de restul datelor, independent de context, și reprezintă cel mai frecvent studiat tip. A doua categorie, anomaliile contextuale, sunt instanțe care sunt anormale într-un anumit context, dar nu global - un volum ridicat de trafic în orele de lucru poate fi normal, în timp ce același volum la ore târzii devine suspect. A treia categorie, anomaliile colective, apare atunci când un grup de instanțe individuale formează împreună un tipar anormal, chiar dacă fiecare instanță în parte nu ar fi considerată anomalie.

Anomaliile punctuale sunt cel mai direct aplicabile în detecția intruziunilor: un host care generează brusc un volum de pachete de zece ori mai mare decât media sa istorică este un exemplu tipic [18]. Algoritmii studiați în această lucrare - Isolation Forest și One-Class SVM - sunt proiectați în principal pentru detecția anomaliilor punctuale, construind o frontieră în spațiul caracteristicilor care separă comportamentul normal de cel deviant.

Anomaliile colective sunt deosebit de relevante pentru detectarea atacurilor de tip port scanning sau a atacurilor distribuite, unde nici un flux individual nu este neapărat anormal, ci tocmai multiplicitatea și coordonarea lor constituie indicatorul de amenințare [14]. Detectarea acestui tip de anomalie necesită analiză temporală sau agregare pe ferestre de timp, ceea ce reprezintă o direcție de extindere naturală a sistemului prezentat în această lucrare.

4.2.3 Aplicații în securitate

Detecția anomaliilor reprezintă fundamentul teoretic al categoriei de IDS bazate pe comportament (anomaly-based IDS) discutate în capitolul anterior, și are aplicații extinse în multiple domenii ale securității cibernetice [2, 18]. Domeniile principale de aplicare includ detecția intruziunilor în rețele, detectarea fraudelor financiare, identificarea programelor malițioase prin analiza comportamentului și detectarea amenințărilor persistente avansate (APT), unde atacatorul rămâne ascuns în rețea perioade îndelungate.

În detecția intruziunilor în rețele, abordările bazate pe anomalii prezintă avantajul esențial de a putea identifica atacuri noi și necunoscute la momentul antrenării, inclusiv variante zero-day, în contrast cu sistemele bazate pe semnături [14]. Modelul învață o reprezentare a traficului normal și semnalează orice deviere semnificativă, indiferent dacă acea deviere corespunde unui

tip de atac cunoscut sau nu. Această proprietate este tocmai motivația principală pentru utilizarea tehnicilor de machine learning în sistemul propus în lucrarea de față.

Principalele provocări ale aplicării detecției anomaliilor în securitate sunt concept drift-ul (evoluția în timp a comportamentului normal, care face modelul să devină gradual depășit), dezechilibrul de clase (evenimentele de atac reprezintă o fracție mică din traficul total), și interpretabilitatea scăzută a modelelor, care face dificilă justificarea alertelor față de operatorii de securitate [18]. Aceste provocări sunt abordate în detaliu în secțiunile ulterioare ale acestui capitol, respectiv în analiza rezultatelor experimentale din capitolul dedicat aplicației.

4.3 Algoritmi utilizați

Alegerea algoritmului de machine learning reprezintă o decizie centrală în proiectarea unui sistem IDS bazat pe detecția anomaliilor. Această secțiune prezintă cei trei algoritmi evaluați în contextul acestei lucrări: Random Forest, One-Class SVM și Isolation Forest [18]. Dintre aceștia, Random Forest constituie nucleul sistemului implementat, datorită performanței ridicate pe fluxuri de rețea etichetate, robustеții la dezechilibrul de clase și interpretabilității scorurilor returnate de `predict_proba`.

4.3.1 Random Forest

Random Forest este o metodă de tip ansamblu introdusă de Breiman (2001), care construiește un număr mare de arbori de decizie pe subseturi aleatoare ale datelor de antrenare și combină predicțiile acestora prin vot majoritar (clasificare) sau medie (regresie) [19]. Fiecare arbore este antrenat pe un eșantion bootstrap din date, iar la fiecare nod partiționarea se alege dintr-un subset aleator de caracteristici - două niveluri de aleatorism care reduc varianța și îmbunătățesc generalizarea față de un arbore de decizie singular. În contextul IDS, Random Forest se aplică tipic în regim supervizat, atunci când sunt disponibile fluxuri de trafic etichetate atât ca normale, cât și ca atacuri.

Un avantaj important al algoritmului este că produce scoruri de importanță a caracteristicilor ca produs secundar al antrenării, identificând care atribute ale fluxurilor de rețea contribuie cel mai mult la distingerea traficului normal de cel malițios [19]. Această informație ajută analiștii să înțeleagă de ce un anumit flux a fost marcat ca malițios, ceea ce este util în investigarea incidentelor. Pe dataset-ul CICIDS2017, Random Forest obține în mod consecvent acurateță ridicată (peste 99% pentru anumite tipuri de atac), ceea ce îl face un baseline solid pentru detecția supervizată a intruziunilor.

Limitarea fundamentală a Random Forest în raport cu scopul acestei lucrări este caracterul său supervizat, care impune existența unor date de antrenare etichetate acoperind toate tipurile de atac de interes [15]. Prin urmare, modelul nu poate detecta atacuri noi sau variante neprevăzute la momentul antrenării. Din această cauză, Random Forest este evaluat în această lucrare ca un

punct de referință comparativ față de metodele bazate pe anomalii, care pot opera fără etichete exhaustive de atac.

4.3.2 One-Class SVM

One-Class SVM este o variantă a mașinilor cu vectori suport proiectată pentru detecția noutăților (novelty detection), introdusă de Schölkopf et al. (1999), care învață o descriere compactă a unei singure clase (traficul normal) în locul unei frontiere între două clase [20]. Algoritmul identifică în spațiul de caracteristici indus de o funcție kernel (tipic RBF) o hipersferă de volum minim care conține majoritatea instanțelor de antrenare. Instanțele noi situate în afara acestei hipersphere sunt clasificate ca anomalii.

Hiperparametrul $\nu \in (0, 1]$ controlează compromisul dintre fracțiunea de outlieri tolerați în datele de antrenare și volumul hiperspherei de decizie, jucând un rol similar parametrului de contaminare din metodele bazate pe densitate [16]. One-Class SVM este o abordare semi-supervizată în sensul că se antrenează exclusiv pe exemple din clasa țintă (trafic normal), fără a necesita instanțe etichetate de atac, ceea ce îl face teoretic aplicabil în medii operaționale unde traficul malițios etichetat este rar.

Principala limitare a One-Class SVM pentru monitorizarea rețelelor de înaltă viteză este complexitatea computațională la inferență: timpul de predicție crește pătratic cu dimensiunea setului de antrenare în cazul defavorabil, iar alegerea kernelului și a parametrului ν necesită validare atentă pentru a evita supraajustarea sau un număr excesiv de fals-pozitive [20]. Aceste caracteristici fac algoritmul mai puțin adecvat decât Isolation Forest pentru clasificarea în timp real a fluxurilor de înaltă viteză.

4.3.3 Isolation Forest

Isolation Forest, introdus de Liu, Ting și Zhou (2008), este un algoritm de detecție a anomaliilor bazat pe principiul că instanțele anomale sunt puține la număr și structural diferite de cele normale, ceea ce le face mai ușor de izolat prin partiționare aleatorie recursivă [21]. Algoritmul construiește un ansamblu de arbori de izolare (iTree), fiecare construit selectând aleator o caracteristică și o valoare de partiționare în intervalul acesteia. Instanțele anomale, fiind structural izolate, necesită în medie mai puțini pași de partiționare pentru a fi separate și prezintă astfel lungimi de cale mai scurte în arbori.

Scorul de anomalie al unei instanțe este derivat din lungimea medie a căii sale în toți arborii din ansamblu, normalizată față de lungimea de cale așteptată pentru o instanță aleatoare într-un arbore de căutare binar de aceeași dimensiune [21]. Instanțele cu lungime de cale semnificativ mai scurtă decât valoarea așteptată primesc scoruri mari și sunt marcate ca anomalii. Scorul este remapat în intervalul $[0, 1]$ folosind formula $\text{score} = \text{clip}(0.5 - \text{raw_score}, 0, 1)$, iar decizia binară este luată prin compararea cu pragul implicit al parametrului de contaminare.

În implementarea dezvoltată pentru această lucrare, modelul este configurat cu `n_estimators=200` arbori, `max_samples=512` instanțe per arbore și `contamination=0.01`, reflectând proporția de aproximativ 1% fluxuri de atac prezentă în dataset-ul CICIDS2017 [12]. Modelul este încapsulat într-un pipeline scikit-learn ce include un pas de normalizare `StandardScaler`, asigurând stabilitate numerică pentru caracteristici cu scale diferite (de exemplu, durata fluxului în secunde față de numărul de pachete). Antrenarea se realizează exclusiv pe clasa BENIGN din CICIDS2017 (maximum 300 000 fluxuri, cu valori extreme trunchiate la percentila 99.9), iar modelul persistent este salvat în format joblib și încărcat la pornirea serviciului FastAPI.

4.4 Feature engineering în trafic de rețea

Feature engineering este procesul de transformare a datelor brute de rețea într-o reprezentare numerică structurată, adecvată pentru algoritmii de machine learning [11]. Calitatea caracteristicilor determină în mod direct capacitatea modelului de a distinge traficul normal de cel malițios: caracteristici slab alese pot face imposibilă separarea claselor chiar și pentru cei mai puternici algoritmi. În contextul acestei lucrări, setul de caracteristici a fost definit pe baza coloanelor disponibile în CICIDS2017 și aliniat cu outputul instrumentului CICFlowMeter, rezultând un vector de 11 atribute numerice.

4.4.1 Caracteristici statistice

Caracteristicile statistice sintetizează distribuția volumului și dimensiunii pachetelor dintr-un flux de rețea [11, 12]. Ele capturează proprietățile agregate ale comunicației, fără a impune o ordonare temporală a pachetelor, ceea ce le face robuste la variațiile de latență și reordonarea pachetelor specifice rețelelor reale.

Sistemul utilizează cinci caracteristici statistice: numărul total de pachete trimise în direcția forward (`total_fwd_packets`) și backward (`total_bwd_packets`), volumul total de octeți în fiecare direcție (`total_fwd_bytes`, `total_bwd_bytes`) și dimensiunea medie a pachetelor din flux (`avg_packet_size`). Asimetria dintre direcțiile forward și backward este deosebit de informativă: un flux DoS tipic are un raport mare de pachete forward față de backward, deoarece atacatorul trimite volume masive de trafic fără a aștepta răspunsuri de dimensiune comparabilă [12].

4.4.2 Caracteristici temporale

Caracteristicile temporale surprind dinamica comunicației de rețea de-a lungul timpului și sunt esențiale pentru detectarea atacurilor care se manifestă prin comportament anormal de viteză sau durată [12]. Spre deosebire de caracteristicile statistice care agregă volume absolute, cele temporale normalizează activitatea la unitatea de timp, oferind o perspectivă comparabilă

între fluxuri de lungimi diferite.

Sistemul folosește trei caracteristici temporale: durata fluxului în secunde (`flow_duration`, convertită din microsecunde cum sunt stocate în CICIDS2017), rata de octeți pe secundă (`flow_bytes_per_sec`) și rata de pachete pe secundă (`flow_packets_per_sec`). Un atac DoS Hulk, de exemplu, prezintă valori extreme pentru `flow_packets_per_sec` combinate cu o durată scurtă, un tipar complet diferit față de traficul HTTP normal care are rate moderate și durată variabilă [11].

4.4.3 Caracteristici bazate pe flux

Caracteristicile bazate pe flux descriu proprietățile structurale ale conexiunii de rețea și comportamentul protocolului de transport [11]. Fluxul bidirecțional este definit prin 5-tuplu (IP sursă, IP destinație, port sursă, port destinație, protocol) și agregă toate pachetele schimbate între aceiași doi endpoint-uri pe durata sesiunii.

Sistemul utilizează trei caracteristici de acest tip: portul de destinație (`destination_port`), numărul de flag-uri TCP SYN (`syn_flag_count`) și numărul de flag-uri TCP RST (`rst_flag_count`). Portul de destinație identifică serviciul vizat și este un indicator puternic al intenției: conexiunile repetate spre portul 22 (SSH) sau 3389 (RDP) sugerează atacuri brute-force, în timp ce accesarea secvențială a unui număr mare de porturi diferite indică scanare [12]. Flag-urile SYN și RST sunt relevante pentru detectarea SYN flood-urilor și a scanărilor de tip connect, unde un RST primit la fiecare port testat produce un număr ridicat de reset-uri.

4.5 Probleme specifice ML în securitate

Aplicarea tehnicilor de machine learning în domeniul securității rețelelor ridică provocări specifice care nu apar, sau apar cu intensitate redusă, în domeniile clasice de aplicare a ML [14, 18]. Aceste provocări derivă din natura datelor de trafic de rețea și din caracteristicile mediului operațional al unui sistem IDS: volume mari de date, distribuții extrem de dezechilibrate între clase, evoluție continuă a comportamentului rețelei și costuri asimetrice ale erorilor de clasificare.

4.5.1 Dezechilibru de clase

Dezechilibrul de clase reprezintă situația în care unele categorii de date sunt reprezentate cu frecvențe mult mai mici decât altele în setul de antrenare [18]. În contextul IDS, traficul benign depășește în mod normal atacurile cu câteva ordine de mărime, ceea ce determină algoritmii de clasificare supervizați să fie biasați spre clasa majoritară. Un model care clasifică toate fluxurile drept BENIGN obține acuratețe aparent ridicată, dar are rată de detecție zero.

În dataset-ul CICIDS2017 utilizat în această lucrare, dezechilibrul este sever: clasa BENIGN reprezintă 80.3% din date (2.271.205 fluxuri), în timp ce clase precum Heartbleed (11 fluxuri) sau Web Attack SQL Injection (21 fluxuri) sunt practic neglijabile [12]. La polul opus, DoS Hulk (8.1%), PortScan (5.6%) și DDoS (4.5%) sunt suficient de reprezentate pentru antrenare efectivă. Pentru a compensa dezechilibrul, modelul Random Forest este configurat cu `class_weight="balanced"`, care ponderează automat contribuția fiecărei instanțe invers proporțional cu frecvența clasei sale [19].

4.5.2 Concept drift

Concept drift desemnează fenomenul prin care distribuția statistică a datelor de intrare sau relația dintre caracteristici și etichete se modifică în timp, determinând degradarea treptată a performanței unui model antrenat pe date istorice [23]. În securitatea rețelelor, acest fenomen este deosebit de pronunțat: comportamentul traficului normal evoluează odată cu adoptarea de noi aplicații și protocoale, iar atacatorii își adaptează continuu tehnicile pentru a eluda sistemele de detecție existente.

Tipurile de concept drift relevante pentru IDS includ drift-ul gradual (schimbări lente ale comportamentului normal pe parcursul săptămânilor sau lunilor, cauzate de actualizări software sau modificări ale obiceiurilor utilizatorilor) și drift-ul brusc (apariția unui nou tip de atac sau a unei noi variante de malware care nu a fost văzut în antrenare) [23]. Sistemul implementat nu include un mecanism automat de detecție a drift-ului, dar arhitectura permite reantrenarea manuală prin endpoint-ul `POST /train` sau prin rularea scriptului de antrenare cu date actualizate, atunci când administratorul observă degradarea performanței.

4.5.3 False positives în context ML

Rata fals-pozitivelor reprezintă proporția instanțelor benigne clasificate eronat ca atacuri de către model, și este una dintre cele mai critice metrici operaționale pentru un sistem IDS [14]. Un model cu precizie scăzută generează un volum mare de alerte false, producând fenomenul de alert fatigue și reducând capacitatea echipei de securitate de a se concentra pe amenințările reale.

Din perspectiva machine learning, există un compromis fundamental între sensibilitate (recall) și specificitate (precizie) controlat prin pragul de decizie al modelului [16]. Reducerea pragului crește recall-ul dar și numărul de fals-pozitive; ridicarea pragului produce efectul opus. Random Forest gestionează mai bine acest compromis față de metodele nesupervizate, datorită probabilităților calibrate returnate de `predict_proba`: scorul de anomalie este calculat ca $1 - P(\text{BENIGN})$, permițând ajustarea fină a pragului de alertare fără reantrenare. Rezultatele experimentale obțin precizie de 0.9368 și recall de 0.9974 pentru Random Forest, demonstrând că dezechilibrul dintre tipurile de erori este gestionabil în practică.

4.6 Dataset-uri pentru IDS

Dataset-urile pentru evaluarea sistemelor IDS sunt colecții de capturi de trafic de rețea etichetate, care conțin atât trafic normal cât și atacuri generate în medii controlate [14]. Absența unor astfel de seturi de date publice de referință a reprezentat mult timp un obstacol major în compararea obiectivă a metodelor de detecție, deoarece fiecare echipă de cercetare utiliza propriile capturi, imposibil de comparat cu altele.

4.6.1 NSL-KDD

NSL-KDD este o versiune îmbunătățită a dataset-ului KDD Cup 99, creat pentru a remedia limitările majore ale predecesorului său, în special prezența masivă a înregistrărilor duplicate care distorsionau evaluările de performanță [22]. Dataset-ul conține 125.973 de înregistrări de antrenare și 22.544 de testare, cu patru categorii de atacuri: DoS, Probe (scanare), R2L (Remote-to-Local) și U2R (User-to-Root). NSL-KDD a devenit un benchmark standard în literatura de specialitate pentru decada 2010–2020, deși caracteristicile sale sunt derivate din date sintetice generate în 1998 și nu mai reflectă traficul de rețea modern.

Principala limitare a NSL-KDD este faptul că atributele sale (41 de caracteristici din date de 1998) nu sunt direct compatibile cu traficul actual [22]. Modelele antrenate pe NSL-KDD prezintă adesea performanțe artificiale ridicate care nu se generalizează la trafic real modern, ceea ce a motivat adoptarea CICIDS2017 ca dataset principal în această lucrare.

4.6.2 CICIDS2017

CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) este un dataset generat prin capturarea traficului real de rețea pe parcursul unei săptămâni de lucru, cu atacuri executate controlat în a doua jumătate a fiecărei zile, caracteristicile fiind extrase cu CICFlowMeter [12]. Setul de date utilizat în această lucrare conține opt fișiere CSV cu un total de 2.830.743 de fluxuri: traficul de luni conține exclusiv trafic benign (529.918 fluxuri), iar zilele de marți până vineri conțin atacuri specifice -FTP/SSH Patator (marți), DoS și Heartbleed (miercuri), Web Attacks și Infiltration (joi), PortScan și DDoS (vineri).

Distribuția claselor după curățare reflectă dezechilibrul tipic al traficului real: BENIGN 80.3% (2.271.205 fluxuri), DoS Hulk 8.1%, PortScan 5.6%, DDoS 4.5%, urmate de clase minoritare (Bot, Web Attacks, Heartbleed) cu sub 0.1% fiecare [12]. Față de NSL-KDD, CICIDS2017 prezintă avantajele unui trafic capturat în condiții realiste, tipuri de atacuri moderne și compatibilitate directă cu instrumentele de captură în timp real, ceea ce l-a consacrat drept standard de facto pentru evaluarea sistemelor IDS bazate pe ML în literatura recentă.

4.6.3 Alte dataset-uri

Pe lângă NSL-KDD și CICIDS2017, literatura de specialitate folosește o serie de alte dataset-uri pentru evaluarea sistemelor IDS [14]. UNSW-NB15, creat de Universitatea din New South Wales în 2015, conține 2.5 milioane de înregistrări cu nouă tipuri de atacuri moderne (backdoors, fuzzing, worms) și prezintă avantajul unui trafic mai reprezentativ pentru amenințările contemporane față de KDD Cup 99. CIC-IDS-2018 este succesorul direct al CICIDS2017, generat de același grup de cercetare cu o infrastructură extinsă și tipuri suplimentare de atacuri, inclusiv atacuri asupra sistemelor financiare și botnet-uri.

Alegerea dataset-ului adecvat pentru o aplicație IDS depinde de tipurile de atacuri relevante pentru mediul țintă, de compatibilitatea caracteristicilor cu pipeline-ul de captură și de dimensiunea necesară pentru antrenarea modelelor selectate [14]. CICIDS2017 a fost ales în această lucrare datorită compatibilității directe cu caracteristicile exportate de CICFlowMeter și cu cele calculate de senzorul Scapy implementat, asigurând coerența între datele de antrenare și traficul evaluat în timp real.

4.7 Preprocesarea datelor

Preprocesarea datelor este etapa prin care datele brute extrase din fișierele CICIDS2017 sunt transformate într-o formă numerică curată, consistentă și scalată, gata pentru antrenarea modelului de machine learning [15]. Calitatea acestei etape influențează direct performanța modelului: date cu valori lipsă netratate, scale extreme necompensate sau caracteristici irelevante incluse pot degrada semnificativ capacitatea de generalizare.

4.7.1 Curățare date

Curățarea datelor implică eliminarea sau corecția înregistrărilor care conțin valori invalide sau inconsistente [15]. În cazul CICIDS2017, principalele probleme identificate sunt valorile infinite și valorile lipsă generate de calculul caracteristicilor temporale pentru fluxuri cu durată zero: împărțirea la zero produce valori `Inf` sau `-Inf` pentru `flow_bytes_per_sec` și `flow_packets_per_sec`.

Implementarea aplică trei pași secvențiali: înlocuirea valorilor `Inf` și `-Inf` cu `NaN`, eliminarea rândurilor cu `NaN` pe oricare din cele 11 caracteristici și filtrarea fluxurilor cu durată zero sau negativă. Din cele 2.832.876 de rânduri inițiale, 2.867 (0.10%) au fost eliminate, un procent neglijabil care nu afectează echilibrul de clase. Suplimentar, valorile extreme ale caracteristicilor de volum și rată sunt trunchiate la percentila 99.9 pentru fiecare coloană, prevenind distorsionarea scalerului de către outlieri excepționali, cum ar fi fluxurile DoS cu milioane de octeți pe secundă [18].

4.7.2 Normalizare

Normalizarea transformă caracteristicile la scale comparabile, eliminând influența diferențelor de magnitudine asupra algoritmilor sensibili la scală [16]. Fără normalizare, caracteristici cu valori mari (de exemplu `total_fwd_bytes` cu valori de ordinul milioanelor) ar domina distanțele față de caracteristici cu valori mici (de exemplu `syn_flag_count` cu valori 0–5).

Sistemul utilizează `StandardScaler` din scikit-learn, care transformă fiecare caracteristică x la media zero și deviația standard unitară: $x' = (x - \mu)/\sigma$, unde μ și σ sunt calculate exclusiv pe setul de antrenare [16]. Scalerul este încapsulat în obiectul `Pipeline` împreună cu clasificatorul, garantând că parametrii μ și σ sunt estimați o singură dată pe datele de antrenare și aplicați consistent la inferență, fără risc de data leakage.

4.7.3 Selecția caracteristicilor

Selecția caracteristicilor este procesul de identificare a subsetului de atribute din setul complet disponibil care contribuie cel mai mult la performanța modelului, eliminând redundanța și zgomotul [15]. CICIDS2017 conține 84 de caracteristici originale generate de CICFlowMeter, din care sistemul utilizează un subset de 11, selectate pe baza a trei criterii: disponibilitate în captura în timp real (caracteristica trebuie să fie calculabilă de senzorul Scapy din pachete individuale), relevanță demonstrată pentru tipurile de atac vizate și absența corelației puternice cu alte caracteristici incluse.

Caracteristicile excluse includ atribute statice (adrese IP, porturi sursă) care nu generalizează la instanțe noi, atribute cu variabilitate insuficientă și atribute corelate cu cele deja incluse [11]. Importanța caracteristicilor poate fi evaluată post-hoc prin atributul `feature_importances_` al modelului Random Forest antrenat, care oferă o estimare bazată pe reducerea medie a impurității Gini în arborii de decizie.

4.8 Metrici de evaluare

Evaluarea performanței unui sistem IDS bazat pe machine learning necesită metrici cantitative care să captureze diferite aspecte ale calității clasificării [16]. Folosirea unei singure metrici poate induce concluzii eronate, în special în prezența dezechilibrului de clase: acuratețea globală poate fi ridicată chiar dacă modelul ratează marea majoritate a atacurilor. Prin urmare, evaluarea utilizează un set complementar de metrici derivate din matricea de confuzie, care conține patru valori: adevărat-pozitive (TP), fals-pozitive (FP), fals-negative (FN) și adevărat-negative (TN).

4.8.1 Accuracy

Acuratețea măsoară proporția instanțelor clasificate corect din totalul instanțelor evaluate [16]. Formula sa este:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Modelul Random Forest obține o acuratețe globală de 0.99 pe setul de test CICIDS2017. Această valoare este influențată de dominanța clasei BENIGN și trebuie interpretată împreună cu metricile per clasă.

4.8.2 Precision

Precizia măsoară proporția predicțiilor pozitive (anomalii) care sunt cu adevărat atacuri, indicând cât de mulți dintre cei marcați drept atacatori sunt efectiv atacatori [16]. Formula sa este:

$$\text{Precision} = \frac{TP}{TP + FP}$$

O precizie ridicată înseamnă puține fals-pozitive, reducând alert fatigue și costul investigării manuale. Modelul Random Forest obține precizie binară de 0.9368, indicând că 93.7% din alertele generate corespund unor atacuri reale.

4.8.3 Recall

Recall-ul măsoară proporția atacurilor reale care au fost detectate de model, indicând cât de mulți dintre atacatori sunt identificați [16]. Formula sa este:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall-ul ridicat este deosebit de important în aplicații de securitate, unde costul unui atac nedetectat este tipic mult mai mare decât costul unei alerte false. Random Forest obține recall binar de 0.9974, detectând 99.7% din atacurile din setul de test.

4.8.4 F1-score

F1-score este media armonică a preciziei și recall-ului, oferind o metrică unică ce balansează cele două [16]. Formula sa este:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Media armonică penalizează puternic valorile mici: un model cu precizie 1.0 și recall 0.0 obține F1 = 0. F1-score este metrica principală de comparare a celor trei modele evaluate:

Random Forest (0.9662), One-Class SVM (0.5820) și Isolation Forest (0.0012) pe setul de test CICIDS2017.

4.8.5 ROC-AUC

Curba ROC (Receiver Operating Characteristic) vizualizează performanța unui clasificator binar pentru toate valorile posibile ale pragului de decizie, prin reprezentarea ratei de adevărat-pozitive față de rata de fals-pozitive [16]. Aria de sub curbă (AUC) sintetizează această performanță într-un scalar între 0 și 1, unde AUC = 0.5 corespunde clasificării aleatoare și AUC = 1.0 corespunde separării perfecte.

ROC-AUC este mai puțin sensibilă la dezechilibrul de clase decât acuratețea și permite compararea modelelor independent de alegerea pragului de decizie [16]. Deși nu a fost calculată explicit în experimentele principale ale acestei lucrări, poate fi generată cu `sklearn.metrics.roc_auc_score` și reprezintă o direcție naturală de aprofundare a evaluării.

4.9 Validarea modelelor

Validarea modelelor de machine learning urmărește estimarea cât mai fidelă a performanței pe date nevăzute, adică a capacității de generalizare a modelului antrenat [16]. O validare incorectă - de exemplu, evaluarea pe datele de antrenare sau utilizarea informațiilor din setul de test în procesul de antrenare (data leakage) - produce estimări optimiste care nu se confirmă în deployment.

4.9.1 Train/Test split

Împărțirea train/test constă în separarea dataset-ului în două subseturi disjuncte: un set de antrenare pe care modelul este ajustat și un set de test pe care performanța este estimată o singură dată la final [15]. Cheia corectitudinii metodei este că setul de test nu trebuie folosit în niciun fel în procesul de antrenare sau de selecție a hiperparametrilor.

În implementarea acestei lucrări, dataset-ul CICIDS2017 este împărțit în proporție de 80% antrenare (2.262.301 fluxuri) și 20% testare (565.575 fluxuri), utilizând `train_test_split` cu parametrul `stratify=y` [12]. Stratificarea asigură că fiecare subset menține aceeași distribuție de clase ca setul original, prevenind situația în care clase rare (Heartbleed cu 11 instanțe) ar putea fi absente din setul de test. Parametrul `random_state=42` garantează reproductibilitatea împărțirii.

4.9.2 Cross-validation

Cross-validarea k-fold este o metodă mai robustă de estimare a performanței, în care dataset-ul este împărțit în k subseturi egale, modelul este antrenat de k ori - de fiecare dată pe $k - 1$

folduri și testat pe foldul rămas - iar performanța finală este media celor k evaluări [16]. Această abordare reduce varianța estimării față de un singur train/test split și utilizează mai eficient datele disponibile.

Cross-validarea nu a fost aplicată în experimentele principale ale acestei lucrări din considerente computaționale: cu 2.8 milioane de fluxuri și un Random Forest cu 100 de arbori, un singur ciclu de antrenare necesită câteva minute, iar cross-validarea 5-fold ar multiplica acest timp de cinci ori [19]. Train/test split cu stratificare pe un dataset de această dimensiune oferă o estimare suficient de stabilă, variabilitatea datorată alegerii split-ului specific fiind neglijabilă la numărul de exemple disponibil.

Capitolul 5

Prezentarea aplicației propuse

Capitolul de față prezintă implementarea practică a unui sistem complet de detecție a intruziunilor în traficul de rețea. Scopul principal al aplicației este de a monitoriza în timp real traficul de rețea, de a clasifica fiecare flux utilizând modelul de Machine Learning descris în capitolul anterior și de a genera alerte imediate atunci când este detectată o posibilă amenințare cibernetică.

Aplicația materializează în practică întreg pipeline-ul teoretic prezentat în capitolele precedente: de la capturarea pachetelor brute de rețea, prin extragerea caracteristicilor statistice la nivel de flux, până la clasificarea acestora cu un model Random Forest antrenat pe dataset-ul CICIDS2017. Rezultatele clasificării sunt expuse operatorilor de securitate printr-o interfață web interactivă, cu notificări în timp real prin WebSocket.

Sistemul este structurat ca o arhitectură de microservicii cu patru componente independente, fiecare putând fi pornită, oprită și actualizată separat. Această decizie arhitecturală asigură că reantrenarea modelului AI nu necesită oprirea monitorizării, iar actualizarea interfeței nu perturbă pipeline-ul de analiză.

5.1 Analiza cerințelor

Înaintea descrierii arhitecturii tehnice, este util să definim cerințele pe care sistemul trebuie să le îndeplinească. Aceste cerințe au ghidat deciziile de proiectare descrise în secțiunile următoare.

5.1.1 Cerințe funcționale

Cerințele funcționale descriu comportamentele pe care sistemul trebuie să le expună. Au fost identificate zece cerințe funcționale principale, prezentate în Tabelul 5.1.

ID	Denumire	Descriere
RF-01	Capturarea traficului	Senzorul capturează continuu pachetele de pe interfața de rețea locală prin Scapy.
RF-02	Agregarea fluxurilor	Pachetele sunt grupate în fluxuri bidirecționale identificate prin 5-tuplu.
RF-03	Extragerea caracteristicilor	La fiecare 5 secunde senzorul calculează cei 19 vectori de caracteristici per flux și îi trimite backend-ului.
RF-04	Clasificarea fluxurilor	Backend-ul solicită serviciului AI clasificarea fiecărui flux și primește eticheta de atac și scorul de anomalie.
RF-05	Generarea alertelor	La detectarea unei anomalii se creează o alertă cu tipul atacului, severitatea (CRITICAL / HIGH / MEDIUM / LOW) și IP-urile implicate.
RF-06	Notificări în timp real	Alertele sunt transmise instantaneu clienților frontend prin WebSocket, fără polling.
RF-07	Persistența datelor	Fluxurile, rezultatele clasificării și alertele sunt stocate în PostgreSQL pentru auditare ulterioară.
RF-08	Dashboard de monitorizare	Interfața web afișează statistici agregate ale sesiunii: fluxuri analizate, anomalii detectate și distribuția pe tipuri de atac.
RF-09	Gestionarea alertelor	Operatorul poate filtra alertele după severitate sau tip și le poate marca individual ca rezolvate.
RF-10	Metrici model AI	Interfața web expune precizia, recall-ul și F1-score-ul per clasă ale modelului curent.

Tabela 5.1: Cerințele funcționale ale sistemului Network Anomaly Detection

5.1.2 Cerințe nefuncționale

Cerințele nefuncționale definesc constrângerile de calitate sub care sistemul trebuie să funcționeze, independent de comportamentele funcționale descrise anterior. Au fost identificate șase cerințe nefuncționale corespunzând categoriilor de performanță, fiabilitate, portabilitate,

modularitate, securitate și mentenabilitate.

ID	Denumire	Descriere
RNF-01	Performanță	Latența totală de la capturarea unui pachet până la afișarea alertei corespunzătoare în interfața web nu trebuie să depășească 10 secunde în condiții normale de trafic (sub 1000 pachete/secundă).
RNF-02	Modularitate	Fiecare dintre cele patru componente (senzor, serviciu AI, backend, frontend) trebuie să poată fi repornită sau actualizată independent, fără a întrerupe funcționarea celorlalte componente.
RNF-03	Portabilitate	Sistemul trebuie să funcționeze pe hardware obișnuit de tip laptop sau server de birou, fără acceleratoare hardware specializate, și să poată fi instalat pe sisteme de operare Linux și Windows 10/11.
RNF-04	Fiabilitate	În cazul indisponibilității temporare a serviciului AI, backend-ul trebuie să continue să stocheze fluxurile primite și să încerce reclasificarea lor după restabilirea conexiunii, fără pierdere de date.
RNF-05	Securitate minimă	Comunicarea inter-componente este restricționată la rețeaua locală; senzorul necesită privilegii de administrator pentru capturarea pachetelor la nivel de interfață de rețea.
RNF-06	Mentenabilitate	Reantrenarea modelului AI pe date noi trebuie să fie posibilă prin apelul endpoint-ului `POST /train`, fără modificări ale codului sursă și fără oprirea celorlalte componente.

Tabela 5.2: Cerințele nefuncționale ale sistemului Network Anomaly Detection

5.2 Arhitectura sistemului

5.2.1 Componente principale

Sistemul este alcătuit din patru componente independente care comunică prin interfețe REST și WebSocket:

- Senzorul de rețea - un modul Python care capturează pachete cu biblioteca Scapy, le agregează în fluxuri bidirecționale și trimite periodic vectorii de caracteristici extrase la backend; rulează pe portul implicit al interfeței de rețea locale și nu necesită modificări ale infrastructurii existente;
- Serviciul AI - un server FastAPI, care găzduiește modelul Random Forest antrenat pe CICIDS2017 și expune endpoint-uri REST pentru predicție (`/predict`), reantrenare (`/train`), metrici (`/metrics`) și informații model (`/model/info`);
- Backend-ul - o aplicație Spring Boot care orchestrează pipeline-ul de analiză: primește fluxurile de la senzor, solicită clasificarea de la serviciul AI, persistă rezultatele în PostgreSQL, generează alerte și le transmite în timp real prin STOMP WebSocket;
- Frontend-ul - o aplicație React care afișează statistici, alerte și istoricul sesiunilor de monitorizare, actualizate live prin conexiunea WebSocket.

Relațiile dintre componente și direcția fluxului de date sunt ilustrate în Figura 5.1.

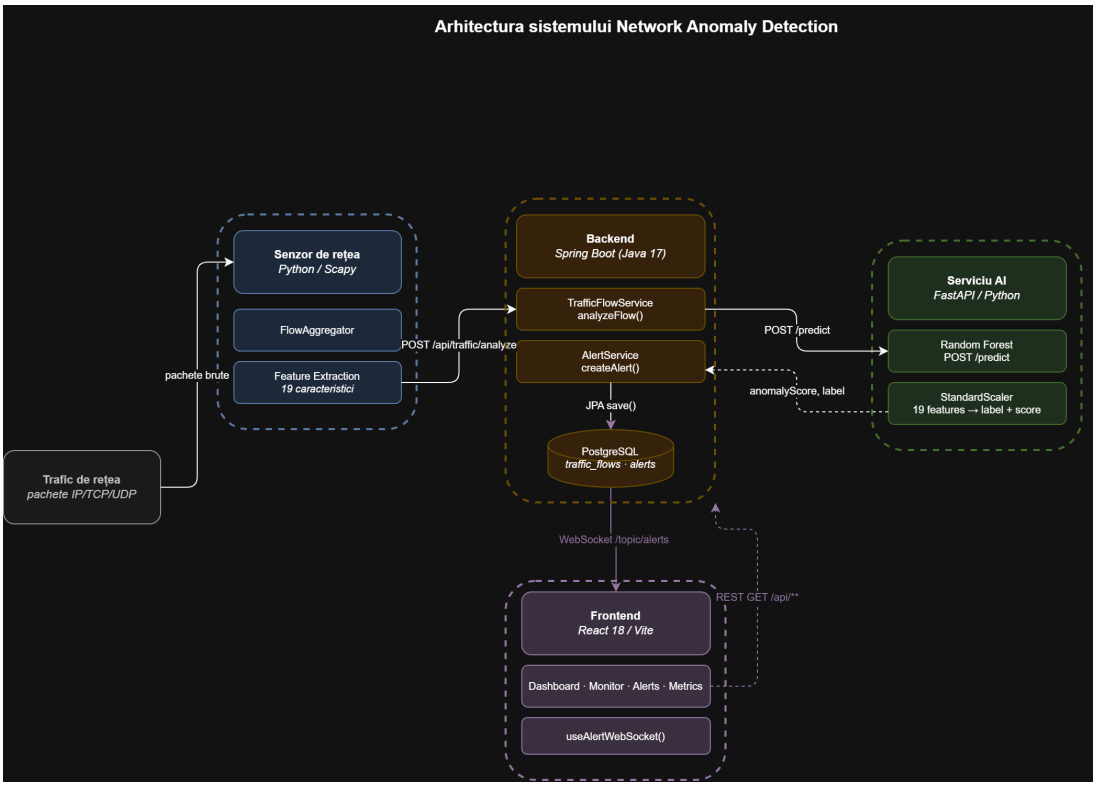


Figura 5.1: Arhitectura de ansamblu a sistemului

5.2.2 Fluxul datelor

Datele traversează sistemul în pași secvențiali, de la captura brută până la alerta afișată în interfața utilizator:

1. Senzorul capturează un pachet de rețea prin Scapy și îl adaugă în structura de date internă (`FlowAggregator`), indexat după cheia canonică a fluxului (5-tuplu bidirecțional).

2. La fiecare 5 secunde (sau imediat la detectarea unui tipar de atac rapid) senzorul apelează `flush()`, calculează cele 19 caracteristici per flux și trimite un `POST /api/traffic/analyze` la backend.

3. Backend-ul salvează fluxul în baza de date PostgreSQL și trimite vectorul de caracteristici la serviciul AI prin `POST /predict`.

4. Serviciul AI scalează caracteristicile cu `StandardScaler` și clasifică fluxul cu modelul Random Forest, returnând un răspuns JSON cu câmpurile `anomaly`, `anomalyScore`, `confidence` și `label`.

5. Backend-ul salvează rezultatul analizei; dacă fluxul este clasificat ca anomalie, creează o alertă și o transmite pe canalul WebSocket `/topic/alerts`.

6. Frontend-ul primește alerta instantaneu prin hook-ul custom `useAlertWebSocket` și actualizează dashboard-ul fără interacțiunea utilizatorului.

5.3 Tehnologii utilizate

Sistemul utilizează Spring Boot (Java 17) pentru backend, cu PostgreSQL ca bază de date și comunicare în timp real prin STOMP WebSocket; React 18 cu Vite pentru interfața utilizator; și FastAPI cu scikit-learn (Python 3.11) pentru serviciul de Machine Learning.

5.4 Senzorul de rețea

5.4.1 Capturarea pachetelor

Senzorul reprezintă punctul de intrare al datelor în sistem, responsabil pentru capturarea traficului de rețea brut și transformarea acestuia în fluxuri structurate. Capturarea se realizează prin biblioteca Scapy, care permite accesul la interfața de rețea la nivel de pachet și analiza headerelor IP, TCP și UDP fără intervenția filtrelor sistemului de operare.

Funcția de capturare este invocată într-un thread dedicat, utilizând `sniff()` din Scapy cu un callback per pachet. Atunci când Scapy nu este disponibil sau privilegiile de administrator lipsesc, senzorul intră automat în modul de simulare, generând un flux sintetic de trafic normal

și anomalii pentru testare. Această decizie de proiectare permite testarea întregului pipeline fără o rețea reală.

5.4.2 Agregarea fluxurilor și extragerea caracteristicilor

Clasa `FlowAggregator` grupează pachetele individuale în fluxuri bidirecționale, menținând o structură de tip dicționar în memorie indexată după cheia canonică a fluxului. Canonicalizarea, prezentată în Fragmentul de cod 5.1, asigură că pachetele din ambele direcții ale aceleiași conexiuni sunt asociate aceluiași intrări, indiferent care capăt inițiază comunicarea.

Fragmentul de cod 5.1: Cheia canonică pentru fluxuri bidirecționale

```
def _canonical_key(self, src_ip, dst_ip, src_port, dst_port, proto):
    if (src_ip, src_port) < (dst_ip, dst_port):
        return (src_ip, dst_ip, src_port, dst_port, proto)
    return (dst_ip, src_ip, dst_port, src_port, proto)
```

Fiecare intrare din dicționar corespunde unui obiect `FlowRecord`, o dataclasă Python care acumulează contori de pachete, octeți, flag-uri TCP și liste de dimensiuni ale pachetelor pe durata ferestrei de agregare. La expirarea intervalului de 5 secunde, metoda `flush()` calculează cele 19 caracteristici din datele acumulate și returnează lista de dicționare gata de transmis la backend.

Un mecanism de detecție rapidă verifică în plus, la fiecare secundă, dacă un flux depășește praguri imediat detectabile: 50 de pachete SYN sau 2 MB de date acumulate declanșează `flush()` anticipat, reducând latența de alertare pentru atacuri de tip SYN Flood sau DoS masiv de la 5 secunde la sub 1 secundă.

5.5 Serviciul de Machine Learning

5.5.1 Preprocesarea și antrenarea modelelor

La pornire, serviciul AI încarcă modelul Random Forest serializat din fișierul `ids_model_rf.joblib` sau, dacă acesta nu există, declanșează antrenarea completă pe datele CICIDS2017 disponibile în directorul `ai/data/`.

Preprocesarea datelor CICIDS2017 implică mai mulți pași secvențiali implementați în `model.py`. Funcția `_clean_df()` redenumește coloanele din formatul CICFlowMeter în nomenclatorul intern, convertește durata fluxului din microsecunde în secunde, elimină rândurile cu valori infinite sau nule și filtrează fluxurile cu durată zero sau negativă (situații rezultate din calculul caracteristicilor temporale pentru conexiuni instantanee). Din cele 2.832.876 de rânduri inițiale, 2.867 (0,10%) au fost eliminate prin această etapă.

Modelul Random Forest este antrenat în cadrul unui `Pipeline` scikit-learn compus din `StandardScaler` și `RandomForestClassifier(n_estimators=100, class_weight="balanced")`.

Parametrul `class_weight="balanced"` compensează dezechilibrul de clase din CICIDS2017, ponderând automat contribuția fiecărei instanțe invers proporțional cu frecvența clasei sale, astfel încât clasele minoritare (Bot, Web Attacks cu câteva sute de instanțe) să nu fie ignorate în procesul de antrenare.

Setul de date este împărțit în 80% antrenare (2.262.208 fluxuri) și 20% testare (565.553 fluxuri) prin `train_test_split` stratificat (parametrul `stratify=y` asigură că fiecare subset menține distribuția de clase din setul complet, evitând situația în care clase rare precum Heartbleed cu 11 instanțe ar fi absente din setul de test).

Pentru referință, sunt antrenate și evaluate suplimentar Isolation Forest (nesupervizat, antrenat exclusiv pe fluxuri BENIGN, maximum 300.000 eșantioane) și One-Class SVM (semi-supervizat, antrenat pe 5.000 de fluxuri BENIGN din considerente de complexitate computațională $O(n^2)$ la inferență).

5.5.2 Endpoint-ul de predicție

Serviciul FastAPI expune endpoint-ul `POST /predict` care primește vectorul de 19 caracteristici și returnează rezultatul clasificării. Formatul schimbului de date este definit prin modele Pydantic, asigurând validarea automată a tipurilor la intrare.

Funcția `predict()` calculează scorul de anomalie ca $1 - P(\text{BENIGN})$ din distribuția de probabilitate returnată de `predict_proba()` a Random Forest, și eticheta de clasă ca argmax al acestei distribuții. Răspunsul conține: `anomaly` (boolean), `anomalyScore` (float în $[0, 1]$), `confidence` (probabilitatea maximă a clasei prezise) și `label` (eticheta de atac: DDoS, PortScan, SSH-Patator etc., sau BENIGN pentru trafic normal).

5.6 Backend-ul aplicației

5.6.1 Pipeline-ul de analiză

Metoda `analyzeFlow()` din clasa `TrafficFlowService` constituie nucleul pipeline-ului de procesare al backend-ului. Aceasta implementează secvența tranzacțională completă: persistare flux, apel AI, persistare rezultat, generare alertă, ca în Fragmentul de cod 5.2.

Fragmentul de cod 5.2: Pipeline-ul de analiză în TrafficFlowService

```
public AnalysisResult analyzeFlow(TrafficFlowRequest request) {
    TrafficFlow flow = buildFlow(request);
    flow = flowRepository.save(flow);

    if (request.getSessionId() != null)
        monitoringService.incrementFlowCount(request.getSessionId());
```

```
    AiPredictionResponse prediction = callAiService(flow);

    AnalysisResult result = new AnalysisResult();
    result.setTrafficFlow(flow);
    result.setAnomaly(prediction.isAnomaly());
    result.setAnomalyScore(prediction.getAnomalyScore());
    result.setConfidence(prediction.getConfidence());
    result.setDetails(prediction.getLabel());
    result.setAnalyzedAt(LocalDateTime.now());
    result = resultRepository.save(result);

    if (prediction.isAnomaly()) {
        alertService.createAlert(flow,
            prediction.getAnomalyScore(),
            prediction.getLabel());
        if (request.getSessionId() != null)
            monitoringService.incrementAnomalyCount(
                request.getSessionId());
    }
    return result;
}
```

Apelul la serviciul AI este realizat prin `RestTemplate`, cu URL-ul configurat prin proprietatea `ai.service.url` din `application.properties`, permițând reconfigurarea ușoară a adresei serviciului fără recompilarea aplicației.

5.6.2 Generarea și clasificarea alertelor

Clasa `AlertService` implementează logica de clasificare a alertelor în două dimensiuni: tipul atacului și severitatea.

Tipul atacului este determinat prioritar din eticheta ML returnată de Random Forest, prin compararea cu șiruri de caractere cunoscute (PORTSCAN, DDOS, SSH-PATATOR, BOT etc.). Dacă eticheta ML este generică sau lipsă, se aplică o rezervă euristică bazată pe caracteristicile fluxului: rate mari de pachete pe secundă indică PORT_SCAN, volume mari de date în timp scurt indică DOS, iar portul destinație 22, 3389 sau 21 sugerează BRUTE_FORCE.

Severitatea este clasificată în patru niveluri pe baza scorului de anomalie returnat de modelul AI: `CRITICAL` pentru scoruri peste 0.8 (probabilitate extrem de ridicată de atac real), `HIGH` pentru intervalul $(0.6, 0.8]$, `MEDIUM` pentru $(0.4, 0.6]$ și `LOW` pentru scoruri sub 0.4, indicând o anomalie posibilă dar cu certitudine redusă.

La finalizarea procesării, alerta persistată în PostgreSQL este transmisă imediat tuturor clienților frontend conectați prin canalul STOMP WebSocket `/topic/alerts`, asigurând vizi-

bilitate instantanee fără polling.

5.7 Interfața utilizator

Interfața utilizator este implementată ca o aplicație React cu cinci pagini accesibile prin React Router v6. Notificările în timp real sunt gestionate de hook-ul custom `useAlertWebSocket`, care stabilește o conexiune STOMP la pornirea componentei și apelează un callback la primirea fiecărei alerte noi, actualizând starea React fără reîncărcarea paginii. Proxy-ul Vite redirecționează cererile `/api` și `/ws` de la frontend la backend, simplificând configurarea CORS în mediul de dezvoltare.

5.7.1 Dashboard

Pagina Dashboard oferă o privire de ansamblu operatorului de securitate: numărul total de fluxuri analizate, anomalii detectate, rata de anomalii și distribuția alertelor pe tipuri de atac prezentate sub formă de statistici agregate. Datele se actualizează automat la fiecare 10 secunde prin apeluri REST la endpoint-urile `GET /api/traffic/stats` și `GET /api/alerts/active`. Interfața Dashboard este prezentată în Figura 5.2.

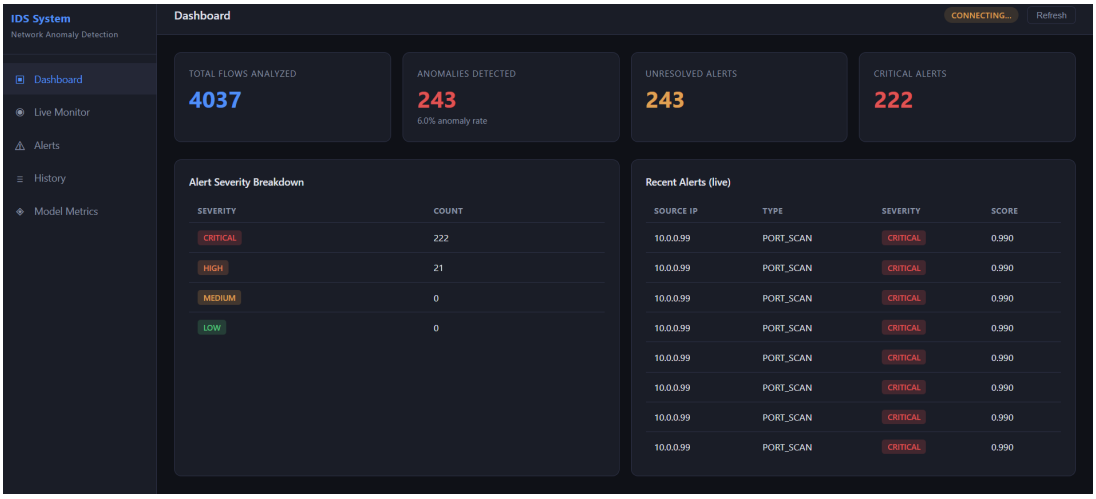


Figura 5.2: Pagina Dashboard cu statistici agregate ale sesiunii de monitorizare

5.7.2 Live Monitor

Pagina Live Monitor afișează în timp real fluxurile de trafic procesate de backend în ultimele 5 minute, sub forma unui tabel paginat. Fiecare rând indică IP-urile sursă și destinație, portul, protocolul, numărul de pachete și rezultatul clasificării. Fluxurile marcate ca anomalii sunt evidențiate vizual, iar coloana de etichetă afișează tipul atacului returnat de modelul Random Forest (DDoS, PortScan, SSH-Patator etc.). Intervalul de reîmprospătare este de 5 secunde,

configurat prin `setInterval` în hook-ul componentei. Interfața Live Monitor este prezentată în Figura 5.3.

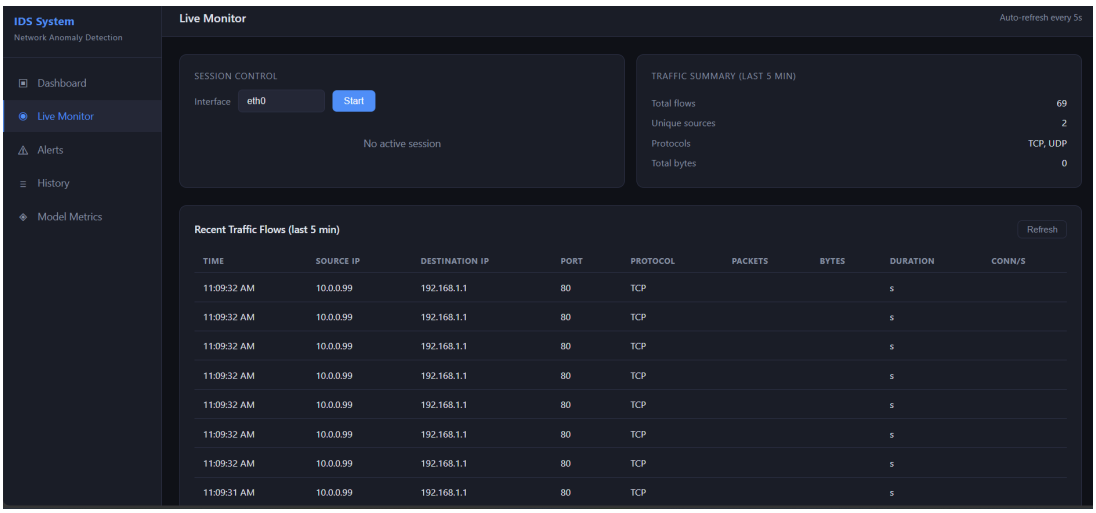


Figura 5.3: Pagina Live Monitor cu fluxuri de trafic în timp real

5.7.3 Gestionarea alertelor

Pagina Alerts afișează lista completă a alertelor generate de sistem, cu posibilitatea de filtrare după severitate (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`) și tip de atac. Fiecare alertă indică IP-urile implicate, portul destinație, scorul de anomalie, tipul clasificat și momentul detectării. Operatorul poate marca o alertă ca rezolvată prin butonul dedicat, ceea ce actualizează câmpul `resolved` în baza de date și elimină alerta din vizualizarea filtrată pe alerte active. Alertele noi sosite prin WebSocket sunt inserate automat în capul listei fără reîncărcarea paginii. Interfața de gestionare a alertelor este prezentată în Figura 5.4.

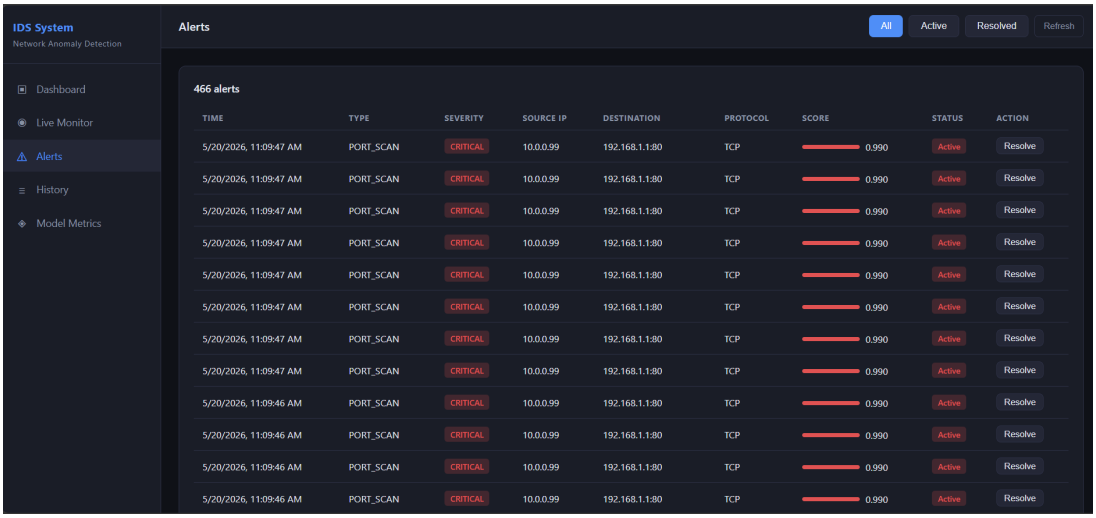


Figura 5.4: Pagina Alerts cu lista alertelor generate și filtrare după severitate

5.7.4 Metricile modelului

Pagina Model Metrics afișează metricile de performanță ale modelului AI curent, citite din endpoint-ul `GET /api/model/metrics`. Sunt prezentate precizia, recall-ul și F1-score-ul per clasă de atac, alături de data antrenării, numărul de caracteristici utilizate și dimensiunile seturilor de antrenare și testare. Pagina History completează tabloul de bord cu istoricul sesiunilor de monitorizare, afișând pentru fiecare sesiune durata, numărul de fluxuri analizate și numărul de anomalii detectate. Pagina Model Metrics este prezentată în Figura 5.5.

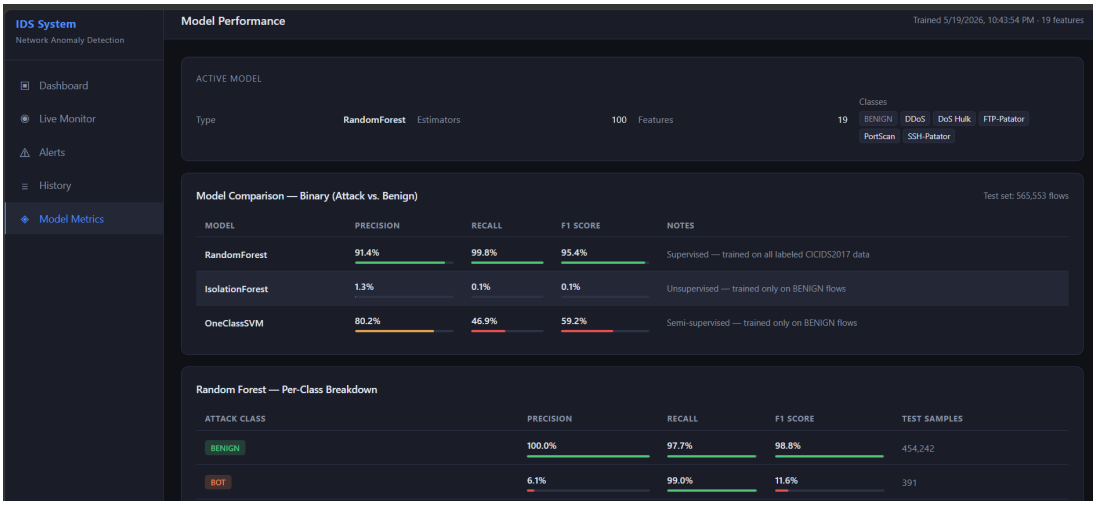


Figura 5.5: Pagina Model Metrics cu performanța Random Forest per clasă de atac

5.8 Evaluarea sistemului

5.8.1 Rezultate experimentale

Evaluarea performanței a fost realizată pe setul de test CICIDS2017, format din 565.553 de fluxuri separate de datele de antrenare printr-un split stratificat 80/20. Modelul a fost antrenat pe 2.262.208 fluxuri, reprezentând toate zilele săptămânii de capturare.

În Tabelul 5.3 sunt prezentate metricile globale pentru cei trei algoritmi evaluați, calculați ca metrici binare (BENIGN față de orice tip de atac):

Model	Precizie	Recall	F1-score
Random Forest	0.9140	0.9981	0.9542
One-Class SVM	0.8015	0.4692	0.5919
Isolation Forest	0.0133	0.0006	0.0011

Tabela 5.3: Comparație metrici globale ale celor trei modele pe setul de test CICIDS2017

Random Forest obține cel mai ridicat F1-score (0.9542), combinând un recall cvasi-perfect (99.81%) cu o precizie solidă (91.40%). Aceasta înseamnă că sistemul detectează 99.8% din

totalul atacurilor prezente în setul de test, cu un fals-pozitiv la fiecare 11.5 alerte în medie.

În Tabelul 5.4 este detaliată performanța Random Forest per clasă de atac:

Clasă	Precizie	Recall	F1	Support
BENIGN	0.9995	0.9770	0.9881	454 242
DDoS	0.9998	0.9995	0.9996	25 605
DoS Hulk	0.9663	0.9977	0.9817	46 025
PortScan	0.9937	0.9995	0.9966	31 761
FTP-Patator	1.0000	0.9994	0.9997	1 587
SSH-Patator	0.8378	0.9983	0.9111	1 180
DoS slowloris	0.9906	0.9957	0.9931	1 159
DoS Slowhttptest	0.9882	0.9873	0.9877	1 100
DoS GoldenEye	0.9610	0.9932	0.9768	2 059
Bot	0.0614	0.9898	0.1156	391
Heartbleed	1.0000	1.0000	1.0000	2
Infiltration	1.0000	0.5714	0.7273	7
Web Attack - Brute Force	0.1303	0.2658	0.1749	301
Web Attack - SQL Injection	0.3333	0.2500	0.2857	4
Web Attack - XSS	0.0418	0.7000	0.0788	130

Tabela 5.4: Performanța Random Forest per clasă de atac pe setul de test CICIDS2017

5.8.2 Analiza performanței

Rezultatele demonstrează comportamente distincte ale modelului în funcție de tipul de atac și reprezentarea sa în datele de antrenare.

Clase cu performanță excelentă. Atacurile volumetrice și cele cu semnătură statistică distinctă obțin F1-score aproape perfect: DDoS (F1 = 0.9996), FTP-Patator (F1 = 0.9997), PortScan (F1 = 0.9966) și DoS Hulk (F1 = 0.9817). Aceste tipuri de atac generează tipare clare în spațiul celor 19 caracteristici: fluxuri cu mii de pachete pe secundă (DoS/DDoS), conexiuni de scurtă durată spre portul 21 cu număr mare de flag-uri ACK (FTP-Patator) sau număr mare de fluxuri SYN de scurtă durată spre porturi distincte (PortScan). Modelul le separă cu ușurință de traficul normal.

Clase cu performanță slabă - dezechilibru sever. Clasele Bot (391 instanțe de test), Web Attack XSS (130) și Web Attack Brute Force (301) prezintă F1-score sub 0.2, în ciuda unui recall relativ ridicat pentru Bot (0.9898) și XSS (0.7000). Precizia extrem de scăzută (0.0614 pentru Bot, 0.0418 pentru XSS) indică un număr mare de fals-pozitive: modelul marchează ca Bot sau XSS numeroase fluxuri care aparțin altor clase, din cauza suprapunerii caracteristicilor în spațiul de 19 dimensiuni. Această limitare este o consecință directă a dezechilibrului de clase din

CICIDS2017: clasa Bot reprezintă aproximativ 0.054% din totalul de 2.827.761 de fluxuri, insuficient pentru a construi o frontieră de decizie precisă, chiar și cu `class_weight="balanced"`.

Comparația cu metodele nesupervizate. Isolation Forest obține un F1-score de 0.0011, practic nul. Acest rezultat se explică prin natura metodei: antrenat exclusiv pe trafic BENIGN, Isolation Forest detectează devieri față de distribuția statistică a traficului normal din CICIDS2017. Tipurile de atac prezente în dataset generează fluxuri care, în spațiul de 19 caracteristici, nu sunt suficient de distincte față de zgomotul natural al traficului benign pentru a fi izolate eficient prin partiționare aleatorie. Aceasta nu înseamnă că metoda este inutilă în general, ci că pentru tipurile de atac bine reprezentate în date etichetate, metoda supervizată este superioară.

One-Class SVM obține un F1-score de 0.5919 cu precizie 0.8015 și recall 0.4692. Precizia ridicată indică că atunci când metoda detectează o anomalie, aceasta este cu probabilitate mare un atac real; recall-ul scăzut înseamnă că sunt detectate mai puțin de jumătate din totalul atacurilor. Această caracteristică îl face adecvat pentru medii unde fals-pozitivele sunt foarte costisitoare, cu acceptarea unui număr mai mare de atacuri nedetectate.

Concluzie. Pentru cazul de utilizare cu date CICIDS2017 etichetate și tipuri de atac cunoscute, Random Forest este alegerea optimă, validând decizia de implementare a acestuia ca model primar în sistemul propus. Performanța sa ridicată pe clasele cu suport suficient (DoS, DDoS, PortScan, Brute Force prin SSH și FTP) demonstrează că sistemul poate detecta cu acuratețe ridicată principalele categorii de amenințări întâlnite în rețelele reale. Limitările identificate pe clasele minoritare sunt documentate și reprezintă direcții concrete de îmbunătățire în versiunile ulterioare.

Capitolul 6

Concluzii teoretice și direcții

Lucrarea de față a urmărit un traseu complet, de la fundamente teoretice până la o implementare funcțională: protocoalele de rețea care generează traficul analizat, mecanismele prin care sistemele IDS detectează amenințările, algoritmii de Machine Learning care stau la baza clasificării, și în final aplicația Network Anomaly Detection System care integrează toate aceste elemente.

6.1 Sinteza metodelor prezentate

Capitolele teoretice au stabilit contextul necesar înțelegerii problemei de detecție a intruziunilor. Modelul TCP/IP și protocoalele de transport definesc structura fluxurilor de rețea ale căror caracteristici statistice sunt extrase de senzor. Clasificarea sistemelor IDS (bazate pe semnătură versus bazate pe anomalie, NIDS versus HIDS) a motivat alegerea abordării bazate pe anomalie statistică, mai adecvată pentru amenințări necunoscute.

Capitolul dedicat Machine Learning a descris cele trei familii de algoritmi evaluate: Random Forest (supervizat), Isolation Forest (nesupervizat) și One-Class SVM (semi-supervizat). Comparația pe dataset-ul CICIDS2017 -2.827.761 de fluxuri etichetate acoperind 15 tipuri de atac reale -a arătat că Random Forest domină cu un F1-score de 0.9542, în timp ce metodele nesupervizate au performanță limitată pe acest dataset datorită naturii distribuției de atribute.

Capitolul de implementare a descris arhitectura de microservicii în patru componente, fluxul de date de la captura Scapy la alerta WebSocket, și deciziile de proiectare care permit reantrenarea sau actualizarea oricărei componente fără oprirea monitorizării.

6.2 Limitări

Dependența de distribuția datelor de antrenare. Modelul a fost antrenat pe CICIDS2017, un dataset capturat în 2017 într-un mediu de laborator. Traficul de rețea real dintr-o organizație contemporană poate diferi semnificativ ca distribuție, ceea ce poate genera atât fals-pozitive

(trafic benign modern clasificat ca atac) cât și fals-negative (atacuri cu profil nou nedetectate).

Concept drift. Profilul traficului rețelelor se modifică în timp; un model antrenat astăzi va degrada gradual în precizie fără reantrenare periodică. Sistemul expune endpoint-ul `POST /train` tocmai pentru a facilita această operație, dar automatizarea sa nu face parte din implementarea curentă.

Clasele cu suport redus. Clasele Bot, Web Attack XSS și Web Attack Brute Force obțin F1-score sub 0.2 datorită dezechilibrului sever din dataset. Atacuri cu câteva sute de instanțe de antrenare nu pot fi modelate precis nici cu `class_weight="balanced"`.

Absența criptării. Comunicarea inter-componente se realizează prin HTTP simplu, adecvat pentru medii de laborator dar insuficient pentru producție. O versiune de producție ar necesita TLS mutual între senzor, backend și serviciul AI.

Senzorul necesită privilegii ridicate. Capturarea pachetelor prin Scapy necesită drepturi de administrator sau echivalent, ceea ce limitează portabilitatea în medii restricționate.

6.3 Testarea sistemului

Corectitudinea componentelor a fost verificată prin patru suite de teste automatizate, acoperind toate componentele sistemului: backend-ul Spring Boot (două suite JUnit 5 + Mockito), serviciul AI FastAPI (pytest + TestClient) și senzorul Python (pytest).

6.3.1 Teste unitare - backend (JUnit 5 + Mockito)

Suita `AlertServiceTest` validează logica de clasificare a alertelor din clasa `AlertService`, izolând dependențele externe prin mock-uri Mockito pentru repository-ul PostgreSQL și template-ul WebSocket. Sunt acoperite patru grupuri: clasificarea severității pe baza scorului de anomalie (pragurile CRITICAL/HIGH/MEDIUM/LOW), maparea etichetelor ML la tipuri de atac (PortScan, DDoS, SSH-Patator, FTP-Patator, Bot, Web Attack), euristicile de rezervă pentru etichete generice și verificarea notificării WebSocket.

Suita `TrafficFlowServiceTest` validează pipeline-ul principal `analyzeFlow()` din `TrafficFlowServi` cu mock-uri pentru toate dependențele (repository-uri, `AlertService`, `MonitoringService`, `RestTemplate`). Sunt acoperite patru grupuri: comportamentul pentru flux benign (nicio alertă, persistență corectă), comportamentul pentru flux anomalie (alertă creată, scor și etichetă transmise corect), integrarea cu `MonitoringService` (contorizarea fluxurilor și anomaliilor per sesiune) și reziliența la căderea serviciului AI (fallback fără excepție propagată).

Rezultatele rulării sunt prezentate în Tabelul 6.1.

Grup de teste	Teste	Trecute	Eșuate	Durată
AlertServiceTest				
Clasificarea severității	5	5	0	20 ms
Tipul alertei din eticheta ML	7	7	0	34 ms
Euristici de rezervă	4	4	0	19 ms
Notificare WebSocket	4	4	0	1319 ms
TrafficFlowServiceTest				
Flux benign	4	4	0	26 ms
Flux anomalie	4	4	0	63 ms
Integrare MonitoringService	4	4	0	68 ms
Reziliență serviciu AI	3	3	0	1861 ms
Total	**36**	**36**	**0**	**3410 ms**

Tabela 6.1: Rezultatele suitelor JUnit 5 + Mockito pentru backend

6.3.2 Teste unitare - senzorul de rețea (pytest)

Suita `test_sensor.py` testează clasa `FlowAggregator` din senzorul Python, fără a necesita o interfață de rețea reală sau biblioteca Scapy. Sunt acoperite patru grupuri: proprietatea de simetrie a cheii canonice (pachetele A→B și B→A aparțin aceluiași flux), contorizarea corectă a pachetelor forward/backward și a flag-urilor TCP, calculul celor 19 caracteristici la `flush()` (inclusiv `avgPacketSize`, `packetLenStd`, `flowBytesPerSec`) și verificarea pragurilor de detecție rapidă.

Rezultatele sunt prezentate în Tabelul 6.2.

Grup de teste	Teste	Trecute	Eșuate	Durată
Cheia canonică	4	4	0	-
Adăugarea pachetelor	7	7	0	-
Flush - calculul caracteristicilor	8	8	0	-
Praguri de detecție rapidă	3	3	0	-
Total	**22**	**22**	**0**	**1.15 s**

Tabela 6.2: Rezultatele suitei test_sensor.py (pytest)

6.3.3 Teste de integrare - serviciul AI (pytest + FastAPI TestClient)

Suita `test_ai_service.py` testează endpoint-urile serviciului FastAPI cu modelul Random Forest real încărcat din disc, utilizând `TestClient` din biblioteca Starlette, care pornește aplicația în același proces fără a necesita un server HTTP real.

Sunt testate șapte scenarii: health check, flux benign nedetectat ca anomalie (scor < 0.5),

flux PortScan detectat cu scor > 0.5, flux DDoS cu rată de 25.000 p/s detectat cu scor > 0.7, flux SSH brute-force detectat ca anomalie, acceptarea payload-ului minimal și respingerea payload-ului invalid (HTTP 422). Un scenariu suplimentar verifică `GET /model/info`: tipul modelului este RandomForest și numărul de caracteristici este exact 19.

Rezultatele sunt prezentate în Tabelul 6.3.

Clasă de test	Teste	Trecute	Eșuate	Durată
TestHealthEndpoint	1	1	0	-
TestBenignFlow	5	5	0	-
TestPortScan	3	3	0	-
TestDDoS	3	3	0	-
TestSSHBruteForce	2	2	0	-
TestMinimalRequest	2	2	0	-
TestModelInfo	3	3	0	-
Total	**19**	**19**	**0**	**3.10 s**

Tabela 6.3: Rezultatele suitei test_ai_service.py (pytest + FastAPI TestClient)

Toate cele patru suite au rulat cu 0 eșecuri. Cele 77 de teste acoperă toate componentele sistemului: senzorul de rețea, serviciul AI, pipeline-ul de analiză din backend și logica de clasificare a alertelor.

6.4 Direcții viitoare

Implementarea curentă reprezintă o bază funcțională care demonstrează fezabilitatea detecției anomaliilor de rețea prin Machine Learning. Pe această bază pot fi construite mai multe extinderi naturale, dintre care cele mai promițătoare sunt descrise în subsecțiunile următoare.

6.4.1 Deep Learning

Modelul Random Forest utilizat în implementarea curentă oferă performanțe ridicate pe dataset-ul CICIDS2017, dar este limitat la clasificarea fluxurilor individuale, tratate ca vectori independenți de caracteristici. O direcție de cercetare semnificativă constă în înlocuirea sau completarea acestuia cu arhitecturi de Deep Learning care pot captura dependențe temporale între fluxuri succesive.

Rețelele recurente de tip LSTM (*Long Short-Term Memory*) sunt adecvate pentru detectarea atacurilor care se desfășoară pe mai multe etape: un port scan urmat de un atac brute-force la câteva minute distanță constituie un pattern temporal pe care un clasificator stateless nu îl poate recunoaște. Autoencoder-ele convoluzionale reprezintă o altă alternativă: antrenate exclusiv pe

trafic benign, reconstruiesc cu eroare redusă fluxurile normale și semnalează ca anomalie orice flux cu eroare de reconstrucție peste un prag calibrat, eliminând dependența de date etichetate.

Ambele abordări necesită volume de date de antrenare semnificativ mai mari și infrastructură de inferență mai costisitoare față de Random Forest, ceea ce justifică decizia de proiectare actuală pentru un sistem de laborator. Trecerea la Deep Learning rămâne relevantă pe măsură ce volumul datelor colectate de senzor crește.

6.4.2 Sisteme în timp real la scară

Arhitectura curentă este proiectată pentru monitorizarea unui singur segment de rețea: un senzor Scapy trimite fluxuri agregatate unui singur backend Spring Boot. Extinderea la o organizație cu mai multe sucursale sau la un mediu cloud necesită o regândire a componentei de colectare și procesare.

O arhitectură distribuită ar înlocui canalul HTTP direct senzor-backend cu o magistrală de mesaje de tip Apache Kafka: fiecare senzor publică fluxuri într-un topic partitionat, iar un cluster de consumatori procesează fluxurile în paralel. Procesarea stateful în timp real, necesară pentru corelarea evenimentelor din surse multiple, ar putea fi realizată cu Apache Flink sau Spark Structured Streaming, care permit definirea de ferestre temporale glisante peste stream-uri distribuite.

Corelarea cross-senzor deschide posibilitatea detectării mișcării laterale: un atacator care scanează mai multe segmente de rețea generează semnale separate pe fiecare senzor, invizibile individual, dar detectabile prin agregare. Stocarea evenimentelor istorice într-o bază de date time-series (InfluxDB, TimescaleDB) ar permite și analiza retrospectivă și calibrarea automată a pragurilor în funcție de profilul de trafic specific fiecărei rețele monitorizate.

În sinteză, lucrarea de față a contribuit cu un sistem funcțional care demonstrează fezabilitatea detecției anomaliilor de rețea prin Machine Learning, integrând captura de pachete, extragerea caracteristicilor, clasificarea supervizată și notificarea în timp real într-o arhitectură de microservicii coerentă. Sistemul se pozitionează ca un punct de plecare concret pentru extinderi în direcțiile enumerate mai sus.

Bibliografie

[1] William Stallings, Network Security Essentials: Applications and Standards, 6th Edition, Pearson.

[2] K. Scarfone, P. Mell, Guide to Intrusion Detection and Prevention Systems (IDPS), NIST Special Publication 800-94, 2007.

[3] https://d1wqtxts1xzle7.cloudfront.net/39755850/retele-libre.pdf?1446837089=&response-content-disposition=inline%3B+filename%3DRetele.pdf&Expires=1775482104&Signature=FrJKIThgpf8UJEXHwpSavQwr0sLmptkpwMJv19qRGqMCJ-YkuEQV7YMJ-4y7JHFR-BpKFWuVcjvTB2zI_&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA, Data accesării: 06.04.2026.

[4] https://ro.wikipedia.org/wiki/Re%C8%9Bea_de_calculatoare, Data accesării: 06.04.2026.

[5] Andrew S. Tanenbaum, Computer Networks, 4th Edition, Pearson Education, 2003.

[6] Behrouz A. Forouzan, Data Communications and Networking, 5th Edition, McGraw-Hill, 2012.

[7] J. Postel, RFC 791: Internet Protocol, IETF, septembrie 1981. Disponibil la: https://www.rfc-editor.org/rfc/rfc791, Data accesării: 01.05.2026.

[8] J. Postel, RFC 793: Transmission Control Protocol, IETF, septembrie 1981. Disponibil la: https://www.rfc-editor.org/rfc/rfc793, Data accesării: 01.05.2026.

[9] J. Postel, RFC 768: User Datagram Protocol, IETF, august 1980. Disponibil la: https://www.rfc-editor.org/rfc/rfc768, Data accesării: 01.05.2026.

[10] GeeksforGeeks, TCP/IP Model, https://www.geeksforgeeks.org/computer-networks/tcp-ip-model/, Data accesării: 05.04.2026.

[11] T. T. Nguyen, G. Armitage, A survey of Techniques for Internet Traffic Classification using Machine Learning, IEEE Communications Surveys & Tutorials, vol. 10, nr. 4, pp. 56–76, 2008.

[12] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization, In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116, 2018.

[13] B. Claise, B. Trammell, P. Aitken, RFC 7011: Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information, IETF, septembrie 2013. Disponibil la: `https://www.rfc-editor.org/rfc/rfc7011`, Data accesării: 05.05.2026.

[14] H. J. Liao, C. H. R. Lin, Y. C. Lin, K. Y. Tung, Intrusion detection system: A comprehensive review, Journal of Network and Computer Applications, vol. 36, nr. 1, pp. 16–24, 2013.

[15] T. M. Mitchell, Machine Learning, McGraw-Hill, 1997.

[16] C. M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.

[17] O. Chapelle, B. Schölkopf, A. Zien (Eds.), Semi-Supervised Learning, MIT Press, 2006.

[18] V. Chandola, A. Banerjee, V. Kumar, Anomaly Detection: A Survey, ACM Computing Surveys, vol. 41, nr. 3, pp. 1–58, 2009.

[19] L. Breiman, Random Forests, Machine Learning, vol. 45, nr. 1, pp. 5–32, 2001.

[20] B. Schölkopf, R. C. Williamson, A. J. Smola, J. Shawe-Taylor, J. C. Platt, Support Vector Method for Novelty Detection, Advances in Neural Information Processing Systems (NIPS), vol. 12, pp. 582–588, 1999.

[21] F. T. Liu, K. M. Ting, Z.-H. Zhou, Isolation Forest, In Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413–422, 2008.

[22] M. Tavallaee, E. Bagheri, W. Lu, A. A. Ghorbani, A Detailed Analysis of the KDD CUP 99 Data Set, In Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), pp. 1–6, 2009.

[23] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A Survey on Concept Drift Adaptation, ACM Computing Surveys, vol. 46, nr. 4, pp. 1–37, 2014.